



SORBONNE UNIVERSITÉ

Master Informatique - STL

RAPPORT DE PROJET – MASTER 1 INFORMATIQUE

Programmation Avancée Fonctionnelle – PAF 2025–2026

Xenon 2 : Megablast

Rapport complet de conception, modélisation sûre et
validation par propriétés

Auteurs

Idris Achabou – Rami Sail



Mai 2026

SORBONNE UNIVERSITÉ
Master Informatique - STL

Résumé

Ce travail présente la conception d'un jeu vidéo sûr en Haskell, structuré autour d'un modèle pur, d'un contrôleur fonctionnel et d'une vue Gloss. Les entités centrales – hitboxes, projectiles, obstacles, ennemis, météores, murs, scores, scripts et moteur – sont accompagnées de contrats explicites. Les invariants éliminent les états dégénérés, les préconditions délimitent le domaine d'appel des opérations et les post-conditions caractérisent les transitions autorisées. L'étude insiste également sur l'usage des structures algébriques pour rendre le comportement composable : le score est traité comme un monoïde, tandis que le script d'événements adopte une structure de monade de liste. La suite de tests HSpec/QuickCheck valide systématiquement ces décisions et met en évidence les choix d'ingénierie ayant permis d'éviter le rejet massif dans les générateurs de données.

Table des matières

1	Introduction	1
2	Manuel d'utilisation et d'exécution	2
2.1	Compilation et exécution	2
2.2	Captures d'écran d'intégration	2
2.3	Matrice des interactions clavier	3
3	Architecture globale et modèle MVC	4
3.1	Flot de contrôle synchrone sous Gloss	4
3.2	Structure concrète du projet et lecture MVC	4
3.3	Façades et stabilité des imports	5
3.4	Architecture de la Vue : fond étoilé pur et déterministe	6
4	Modélisation sûre et contrats de types	7
4.1	Constructeurs intelligents et rejet explicite	7
4.2	Géométrie des hitboxes et murs productifs	7
5	Sûreté algébrique et structures avancées	8
5.1	Score comme monoïde	8
5.2	Script comme monade de liste spécialisée	8
5.3	Preuve des trois lois monadiques pour Script	8
6	Stratégie de test et générateurs constructifs	10
6.1	Proscription de suchThat	10
6.2	Analyse ligne par ligne de genMoteur	10
6.3	Bug détecté : les slots glissants en mode duo	11
6.4	Bug corrigé : murs infinis et ralentissement progressif	11
6.5	Bug corrigé : flux incohérent de projectiles sur Entrée et Espace	12
7	Extensions à haute valeur ajoutée	13
7.1	Mode duo coopératif local	13
7.2	Génération dynamique et rejouabilité	13
7.3	Météores destructibles	14
8	Synthèse quantitative de validation	15
9	Conclusion	16
A	Répertoire exhaustif des contrats implémentés	17
B	Catalogue exhaustif des tests HSpec et QuickCheck	20
C	Code source majeur	31
C.1	SpecHelpers.hs	31
C.2	Step.hs	36
C.3	Collisions.hs	39

Table des figures

1	Écran d'accueil : sélection du mode de jeu et point d'entrée de l'automate applicatif.	2
2	Écran de jeu Solo : instance monojoueur du moteur, avec vaisseau, murs, fond animé et HUD.	3
3	Flux MVC : le modèle reste pur et testable, la boucle Gloss reste confinée dans la couche applicative.	4
4	Écran de jeu Multijoueur : deux joueuses, scores individuels et projectiles signés.	13
5	Écran de Game Over : fin de l'automate de partie et retour contrôlé vers l'accueil.	14
6	Synthèse statistique des tests et propriétés documentés dans le rapport.	15

Liste des tableaux

1	Matrice de routage des événements clavier de l'application.	3
2	Catalogue exhaustif des contrats de sûreté extraits du code source.	17
3	Catalogue exhaustif des tests extraits des fichiers de spécification.	20

Listings

1	Module SpecHelpers.hs : générateurs QuickCheck constructifs sans suchThat	31
2	Module Step.hs : boucle de simulation pure du moteur	36
3	Module Collisions.hs : résolution des impacts et stabilisation des slots joueurs	39

CHAPITRE 1

Introduction

Le projet **Megablast** s'inscrit dans le cadre de l'UE Programmation Avancée Fonctionnelle. Le cahier des charges impose explicitement un développement de jeu vidéo sûr en Haskell, articulé autour de propositions d'invariants, de préconditions, de postconditions, de tests unitaires HSpec et de tests de propriétés QuickCheck. Il attend également une séparation nette entre logique de jeu et effets de bord, ainsi qu'une exploitation raisonnée des classes algébriques du langage.

L'objectif n'est donc pas seulement de produire un programme graphique jouable. Le livrable doit démontrer une discipline de conception : les états incohérents doivent être rejetés à la frontière du modèle, les transitions doivent être formulées comme des fonctions pures, et les propriétés doivent faire partie intégrante de la documentation technique. Cette logique explique l'importance accordée aux annexes : elles ne constituent pas un remplissage, mais le registre vérifiable des engagements contractuels et des tests exécutables du système.

Pour satisfaire aux exigences académiques, l'architecture logicielle s'articule autour de trois axes directeurs :

- **Étanchéité architecturale** : isolation stricte du modèle mathématique pur vis-à-vis des composants d'affichage et de capture événementielle.
- **Génération constructive sans rejet** : conception de générateurs QuickCheck qui synthétisent directement des valeurs valides au lieu de filtrer massivement des valeurs arbitraires.
- **Sémantique algébrique explicite** : recours à Monoid, Functor, Applicative et Monad pour rendre composables les scores, les scripts et les scénarios.

CHAPITRE 2

Manuel d'utilisation et d'exécution

2.1 Compilation et exécution

L'infrastructure de construction repose sur Stack, ce qui garantit un environnement reproductible autour de GHC et des dépendances du projet.

Ligne de commande pour le cycle de développement

```
stack build
stack test
stack run
```

2.2 Captures d'écran d'intégration



Figure 1 – Écran d'accueil : sélection du mode de jeu et point d'entrée de l'automate applicatif.

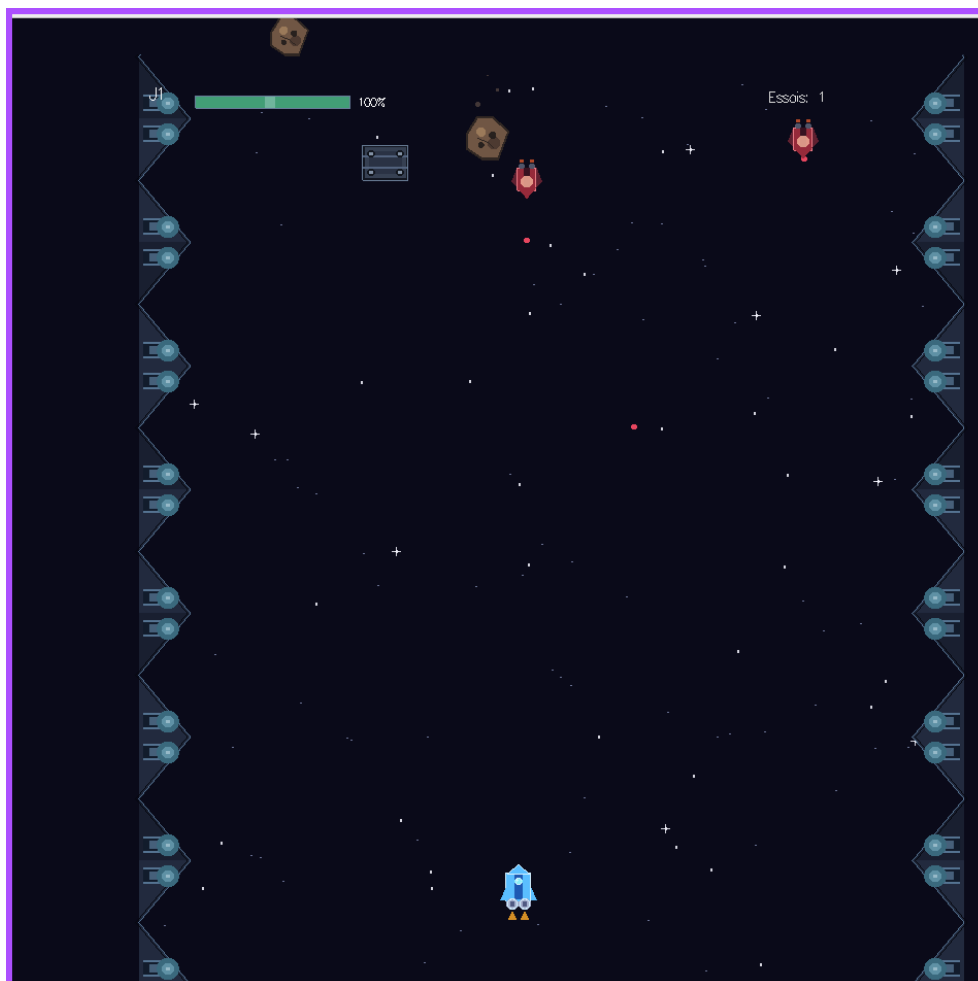


Figure 2 – Écran de jeu Solo : instance monojoueur du moteur, avec vaisseau, murs, fond animé et HUD.

2.3 Matrice des interactions clavier

Contexte applicatif	Commande utilisateur	Effet systémique induit
EcranAccueil	Touche '1'	Initialisation d'une topologie monojoueur.
EcranAccueil	Touche '2'	Instanciation du mode coopératif local.
Partie J1	Flèches directionnelles	Translation géométrique du vaisseau de la première joueuse.
Partie J1	Entrée	Insertion d'un projectile allié signé par l'indice J1.
Partie J2	Z, Q, S, D	Déplacement indépendant de la seconde joueuse.
Partie J2	Espace	Production d'un projectile allié signé par l'indice J2.
Partie	R	Réinitialisation contrôlée du moteur.
Exit	Échap	Quitter le Jeu.

Table 1 – Matrice de routage des événements clavier de l'application.

CHAPITRE 3

Architecture globale et modèle MVC

3.1 Flot de contrôle synchrone sous Gloss

La boucle de jeu principale exploite le point d'entrée `play` de Gloss. Le contrôleur reçoit des événements clavier et des deltas temporels, les transforme en commandes métier, puis délègue toute évolution de la partie au modèle. La vue ne décide jamais de la physique du jeu : elle projette uniquement un état déjà calculé vers une valeur `Picture`.

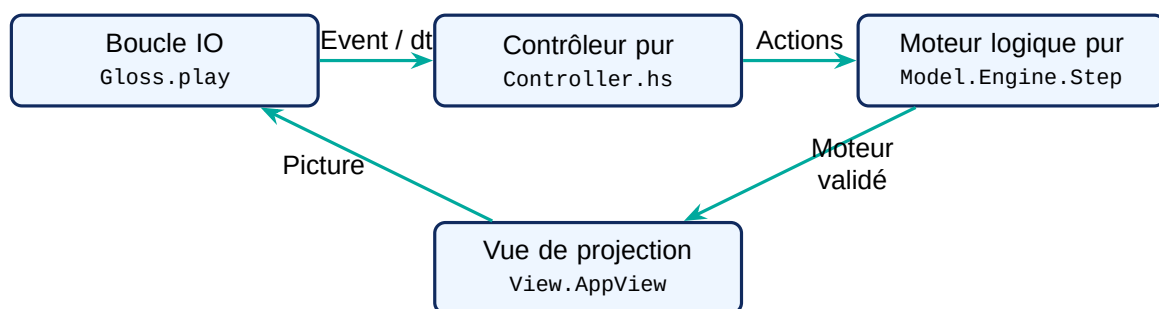


Figure 3 – Flux MVC : le modèle reste pur et testable, la boucle Gloss reste confinée dans la couche applicative.

Cette séparation fournit une testabilité déterministe : les tests injectent directement des structures `Action` et des valeurs `Moteur`, sans fenêtre graphique, sans contexte OpenGL et sans dépendance au temps réel.

3.2 Structure concrète du projet et lecture MVC

L'arborescence du dépôt rend visible la séparation MVC. Le point d'entrée `app/Main.hs` ne contient pas la logique métier : il assemble Gloss, le contrôleur applicatif et la fonction de rendu. Le répertoire `src/Controller` traduit les événements externes en commandes du domaine. Le répertoire `src/Model` contient les types, les invariants, les transitions et les résolutions de collisions. Le répertoire `src/View` transforme l'état courant en valeurs `Picture`, sans recalculer les règles du jeu.

Arborescence fonctionnelle simplifiée

```

MegablastXenon2/
|-- app/
|   |-- Main.hs                entree Gloss / assemblage applicatif
|-- src/
|   |-- Controller/
|   |   |-- ApplicationController.hs    automate ecrans: accueil, partie, game over
|   |   |-- Controller.hs              clavier, cadence, pas de simulation
|   |-- Model/
|   |   |-- Engine.hs                facade publique du moteur
|   |   |-- Engine/
|   |   |   |-- Types.hs              noyau typé du moteur
|   |   |   |-- Step.hs               transition pure de fin de tour
|   |   |   |-- Commands.hs           commandes joueuses
|   |   |   |-- Collisions.hs         resolution des collisions
|   |   |   |-- Murs.hs               defilement et fenetrage des murs
|   |   |   |-- DynamicSpawn.hs       apparition dynamique d'objets
|   |   |   |-- Properties.hs         proprietes transversales
|   |   |   |-- Hitbox.hs              geometrie et collisions elementaires
|   |   |   |-- Objects.hs            obstacles, projectiles, cadences
|   |   |   |-- Meteore.hs            extension meteores
|   |   |   |-- Score.hs              monoide de score
|   |-- View/
  
```


3. Architecture globale et modèle MVC

```
| | |-- AppView.hs           dispatch visuel par etat applicatif
| | |-- Background.hs       fond etoile pur et deterministe
| | |-- MurView.hs          rendu fenetre des murs visibles
| | `-- *View.hs            projections graphiques specialisees
|-- test/
| |-- SpecHelpers.hs        generateurs QuickCheck constructifs
| `-- *Spec.hs              tests HSpec et proprietes QuickCheck
`-- assets/                 sprites, fonds et ressources visuelles
```

Le découpage est volontairement asymétrique : le modèle est plus profond que les deux autres couches, car il concentre les obligations de sûreté. Cette profondeur n'est pas un défaut architectural ; elle permet de factoriser des sous-domaines indépendants : apparition dynamique, intelligence ennemie, collisions ennemies, gestion des murs, commandes et boucle de transition. Les tests peuvent importer directement ces modules ou passer par la façade `Model.Engine`, selon le niveau de granularité recherché.

3.3 Façades et stabilité des imports

Le fichier `src/Model/Engine.hs` joue le rôle de façade publique. Il ne définit presque pas de logique propre : il réexporte les sous-modules `Model.Engine.Types`, `Model.Engine.Step`, `Model.Engine.Commands`, `Model.Engine.Collisions`, `Model.Engine.Murs`, `Model.Engine.DynamicSpawn`, ainsi que les modules périphériques `Model.Score` et `Model.Meteore`. Cette façade a trois intérêts techniques.

- **Interface stable** : les couches `Controller` et `View` importent `Model.Engine` sans connaître le détail de l'organisation interne.
- **Refactorisation locale** : un déplacement de fonction entre `Step.hs`, `Commands.hs` ou `Collisions.hs` ne casse pas nécessairement les imports applicatifs.
- **Frontière MVC lisible** : tout ce qui est exposé par la façade appartient au domaine logique du jeu ; tout ce qui relève du rendu reste dans `View`.

Principe de façade du moteur

```
module Model.Engine
( module Model.Engine.Types
, module Model.Meteore
, module Model.Engine.Murs
, module Model.Engine.Collisions
, module Model.Engine.EnnemiSpawn
, module Model.Engine.Step
, module Model.Engine.Commands
, module Model.Engine.Properties
, module Model.Score
, module Model.Engine.GameConfig
, module Model.Engine.DynamicSpawn
) where
```

Cette organisation protège le reste du programme contre une dépendance excessive aux détails internes. Elle permet également de lire le projet selon une grille simple : `Controller` orchestre, `Model.Engine` décide, `View` dessine. La communication entre les trois couches reste unidirectionnelle : les événements descendent vers le modèle sous forme de commandes, puis l'état validé remonte vers la vue sous forme de données à projeter.

3.4 Architecture de la Vue : fond étoilé pur et déterministe

Le module `Background.hs` fournit un exemple représentatif de vue pure. La liste `etoiles` n'est pas produite par un état mutable. Elle est construite par `genererEtoiles` `graineFondEspace` `nombreEtoiles`, avec une graine fixe 2048 et quatre flux pseudo-aléatoires dérivés : abscisses, ordonnées, tailles et vitesses. Mathématiquement, on peut modéliser ce calcul comme une fonction totale :

$$\text{etoiles} : \mathbb{Z} \times \mathbb{N} \longrightarrow [\text{Etoile}]$$

qui prend une graine et une cardinalité, puis renvoie une liste finie. Une fois cette liste construite, l'animation ne modifie pas les étoiles : elle calcule seulement une ordonnée apparente par congruence :

$$y_t(e) = (e_Y - \lfloor t/e_{vitesse} \rfloor) \bmod H.$$

La parallaxe est donc une projection pure du numéro de tour. Deux appels avec le même tour et la même liste d'étoiles produisent exactement la même image. Cette propriété est cruciale : le rendu donne une impression de mouvement, mais aucune mutation cachée ne vient perturber la reproductibilité des tests du modèle.

CHAPITRE 4

Modélisation sûre et contrats de types

4.1 Constructeurs intelligents et rejet explicite

Le modèle applique le principe selon lequel les états illégaux doivent être rendus inconstructibles ou, à défaut, rejetés explicitement. Les constructeurs intelligents retournent des valeurs `Either Text a`, ce qui force l'appelant à traiter le cas d'échec au lieu de propager silencieusement une donnée invalide.

Encapsulation contractuelle de la cadence

```
data Cadence = Cadence { attente :: Int, restant :: Int } deriving (Eq, Show)

prop_inv_cadence :: Cadence -> Bool
prop_inv_cadence (Cadence a r) = a > 0 && 0 <= r && r < a

mkCadence :: Int -> Either Text Cadence
mkCadence a
  | a > 0 = Right (Cadence a 0)
  | otherwise = Left ('attente doit être > 0')
```

Le tableau de l'Annexe A montre que cette logique est systématique : chaque opération critique possède un domaine d'entrée, un invariant ou une relation de sortie.

4.2 Géométrie des hitboxes et murs productifs

Le système de collision combine points, rectangles, disques, compositions et murs latéraux. Les murs peuvent être représentés par des listes longues ou infinies, d'où la nécessité d'un invariant productif : il contrôle seulement un préfixe borné par `nombrePointsContrôleMur`, ce qui suffit pour garantir l'usage local du moteur sans forcer une structure infinie.

CHAPITRE 5

Sûreté algébrique et structures avancées

5.1 Score comme monoïde

Le score est modélisé par un newtype dont l'opération $\langle \rangle$ correspond à l'addition des points. L'élément neutre est `Score 0`. Cette décision évite de multiplier les fonctions ad hoc : la somme totale d'une liste de scores se réduit à `mconcat`, et la propriété `prop_post_scoreTotal` relie directement l'implémentation au comportement monoïdal attendu.

5.2 Script comme monade de liste spécialisée

Le module `Types.hs` définit :

Structure algébrique du script

```
newtype Script a = Script { scriptToList :: [a] } deriving (Eq, Show)

instance Monad Script where
  Script xs >=> f = Script (concatMap (scriptToList . f) xs)
```

Cette définition donne au script la sémantique de la monade de liste. Un événement peut engendrer zéro, un ou plusieurs événements dérivés ; la liaison monadique concatène ensuite les branches obtenues. Cela correspond naturellement à la construction de scénarios : une vague d'ennemis, une génération procédurale ou une séquence d'actions peuvent être décrites comme des fragments composables.

5.3 Preuve des trois lois monadiques pour script

On raisonne par extensionnalité sur le champ `scriptToList`. Soit $x : a, m = \text{Script } xs, f : a \rightarrow \text{Script } b$ et $g : b \rightarrow \text{Script } c$.

5.3.0.1 Identité gauche. La loi attendue est `pure x >=> f = f x`. Par définition, `pure x = Script [x]`. On calcule :

```
pure x >=> f = Script [x] >=> f
              = Script (concatMap (scriptToList . f) [x])
              = Script (scriptToList (f x))
              = f x.
```

La dernière égalité découle du fait que toute valeur `f x` est nécessairement de la forme `Script ys`.

5.3.0.2 Identité droite. La loi attendue est `m >=> pure = m`. En développant :

```
Script xs >=> pure = Script (concatMap (scriptToList . pure) xs)
                  = Script (concatMap (\x -> [x]) xs)
                  = Script xs.
```

La fonction `concatMap singleton` est l'identité sur les listes ; la structure externe `Script` est donc conservée.

5. Sûreté algébrique et structures avancées

5.3.0.3 Associativité. La loi attendue est $(m \gg= f) \gg= g = m \gg= (\backslash x \rightarrow f x \gg= g)$. En développant le membre gauche :

```
(Script xs >>= f) >>= g = Script (concatMap (scriptToList . f) xs) >>= g
                        = Script (concatMap (scriptToList . g) (concatMap (scriptToList . f) xs)).
```

Le membre droit vaut :

```
Script xs >>= (\x -> f x >>= g) = Script (concatMap (scriptToList . (\x -> f x >>= g)) xs)
                                = Script (concatMap (\x -> scriptToList (f x >>= g)) xs)
                                = Script (concatMap (\x -> concatMap (scriptToList . g) (scriptT
```

Les deux expressions sont égales par associativité de `concatMap` sur les listes. La monade `Script` est donc une spécialisation transparente de la monade de liste, mais typée par le domaine du moteur.

CHAPITRE 6

Stratégie de test et générateurs constructifs

6.1 Proscription de `suchThat`

L'utilisation naïve de `suchThat` sur un état complet du jeu serait inefficace : la probabilité qu'un tirage arbitraire satisfasse simultanément les invariants des hitboxes, des murs, des cadences, des scores, des ennemis et du moteur est trop faible. Le projet adopte donc une génération constructive : chaque sous-générateur produit directement une valeur valide, puis l'état global est assemblé par un constructeur intelligent.

6.2 Analyse ligne par ligne de `genMoteur`

Le générateur central de fuzzing se trouve dans `SpecHelpers.hs`. Sa structure est mathématiquement constructive.

1. `nbObs <- chooseInt (0, 4)` choisit une borne faible pour les obstacles. Cette borne contrôle le coût quadratique potentiel des collisions.
2. `nbProj <- chooseInt (0, 5)` choisit le nombre de projectiles, suffisamment varié pour tester les collisions sans saturer les cas.
3. `nbEnn <- chooseInt (0, 4)` fixe la population ennemie et évite les états artificiellement massifs.
4. `nbMet <- chooseInt (0, 2)` ajoute les météores, avec une cardinalité faible car ils déclenchent des interactions multi-entités.
5. `nbJ <- chooseInt (1, 2)` impose au moins une joueuse et couvre le mode solo ainsi que le mode duo.
6. `vectorOf nbObs genObstacleSimple` ne filtre pas : chaque obstacle est bâti par un générateur de rectangles valides.
7. `vectorOf nbProj genProjectile` produit des projectiles dont propriétaire, direction, hitbox et cadence sont cohérents.
8. `vectorOf nbEnn genEnnemiSimple` assemble des ennemis avec hitbox rectangulaire, PV positifs, oracle et cadence valides.
9. `vectorOf nbMet genMeteoreSimple` produit des météores à rayon positif, cadence valide et dégâts positifs.
10. `vectorOf nbJ genVaisseauActif` garantit que les joueuses générées sont actives : PV positifs et essais disponibles.
11. `cad <- genCadence` produit une cadence globale de scrolling déjà conforme à `prop_inv_cadence`.
12. `evts <- listOf genEvenementPlanifieSimple` génère un script d'événements ; son tri est ensuite pris en charge par le constructeur du moteur.
13. `tour <- chooseInt (0, 30)` force un numéro de tour non négatif, conformément à `prop_inv_moteur`.
14. `seed <- arbitrary` fournit une graine quelconque au générateur pseudo-aléatoire interne ; la validité du moteur ne dépend pas de sa valeur.
15. `mkMoteurTest ...` centralise la validation. Le cas `Right` livre un moteur valide, enrichi par les météores déjà générés. Le cas `Left` n'est pas un filtrage statistique : il joue le rôle d'un garde défensif contre une modification future d'un sous-générateur ou d'un invariant.

Cette construction obtient le bénéfice recherché : les tests explorent des états réalistes et valides au lieu de gaspiller le budget `QuickCheck` en rejets successifs.

6.3 Bug détecté : les slots glissants en mode duo

Stabilisation des identités J1/J2

La première version supprimait physiquement une joueuse éliminée de `mJoueuses`. En mode duo, cela compactait la liste : J2 passait de l'indice 1 à l'indice 0 et héritait donc des commandes et du score de J1. La correction consiste à préserver la longueur de la liste. Une joueuse éliminée devient inactive, mais son slot demeure en place. La propriété `prop_post_supprimerJoueusesEliminees` formalise cette exigence par l'égalité des longueurs avant/après.

6.4 Bug corrigé : murs infinis et ralentissement progressif

Fenêtrage productif des murs

Une difficulté importante provenait de la représentation des murs. Le sujet autorise naturellement des murs décrits par une suite infinie de points, par exemple un mur vertical ou un mur en dents de scie produit récursivement. Cette représentation est élégante sur le plan fonctionnel, mais elle devient dangereuse lorsqu'une opération graphique ou moteur parcourt trop largement la liste. Dans les premières versions, le défilement et certaines translations étaient appliqués sur une portion trop grande du mur. Après plusieurs milliers de tours, environ à partir de 3000 tours de simulation, le jeu commençait à saccader fortement : la fenêtre restait ouverte, mais la partie devenait presque injouable.

La correction consiste à ne conserver et à ne dessiner que la partie utile du mur autour de l'écran. Le moteur applique un *fenêtrage logique* avec une marge de sécurité : les segments définitivement sous l'écran sont éliminés, la liste est coupée lorsqu'elle dépasse le haut visible, puis le haut du mur est complété pour préserver la continuité du niveau. Le module `Model.Engine.Murs` matérialise cette stratégie par la composition suivante.

Normalisation bornée des murs

```
normaliserPointsMur :: [(Int, Int)] -> [(Int, Int)]
normaliserPointsMur =
  completerMurHaut yMaxMurNiveau
  . prendreJusquaHaut yMaxMurNiveau
  . supprimerBasMur yMinMurNiveau
```

Cette définition transforme une structure potentiellement infinie en tranche finie, centrée sur la zone utile du moteur. Le rendu applique la même philosophie dans `View.MurView` : la fonction `segmentsVisibles` ignore les segments placés sous `visibleYMin`, s'arrête dès que le haut visible est dépassé et découpe les segments qui croisent la frontière de l'écran. L'effet est double : la sémantique de mur infini est conservée au niveau conceptuel, mais le coût effectif par image dépend seulement de la hauteur de la fenêtre, et non du nombre total de tours joués.

La propriété `prop_post_defileMurs` documente cette correction : après défilement, les murs restent valides et leur nombre de points est borné par `nombreMaxPointsMur`. Le bug initial était donc un problème de productivité : la liste infinie était correcte mathématiquement, mais elle ne devait jamais devenir l'objet d'un parcours global dans la boucle de jeu.

6.5 Bug corrigé : flux incohérent de projectiles sur Entrée et Espace

Distinction entre touche maintenue et événement ponctuel

Un second bug concernait le tir des joueuses. Les touches Entrée et Espace déclenchaient parfois plusieurs projectiles d'un seul appui, ce qui apparaissait visuellement comme un flux de tir continu. Le problème venait de la confusion entre deux catégories d'actions : les déplacements doivent être répétés tant que la touche reste maintenue, alors que le tir doit être traité comme un événement ponctuel.

La version corrigée sépare explicitement ces deux flux. Les touches de déplacement sont conservées dans `asTouchesPres`, un ensemble de touches actives. En revanche, une touche de tir descendante n'est pas insérée dans cet ensemble ; elle est placée dans `asTirsEnAttente`. Au pas de simulation suivant, ces tirs sont ajoutés seulement aux actions du premier tour moteur, puis la file d'attente est vidée.

Gestion ponctuelle du tir dans le contrôleur

```
gererEvenement (EventKey k Down _ _) as =  
  case toucheVersAction k of  
    Just (i, Tirer) ->  
      as { asTirsEnAttente = asTirsEnAttente as ++ [(i, Tirer)] }  
    _ ->  
      as { asTouchesPres = Set.insert k (asTouchesPres as) }  
  
firstActions = moveActions ++ tirsAttente  
base' = (asfBase asf) { asMoteur = m', asTirsEnAttente = [] }
```

Le point décisif est la fonction `actionsDepuisTouches`. Elle exclut explicitement les actions de tir par `not (estActionTir a)`. Ainsi, même si `simulerStep` exécute plusieurs tours moteur pour rattraper l'accumulateur temporel `Gloss`, le tir n'est injecté qu'au premier tour de cette rafale interne. Les déplacements restent continus, mais les tirs redeviennent des événements discrets. Cette correction rend cohérente la cadence de tir et évite une explosion artificielle de la liste `mProjectiles`.

CHAPITRE 7

Extensions à haute valeur ajoutée

7.1 Mode duo coopératif local

Le mode duo enrichit le modèle sans rompre la pureté de l'architecture. Les projectiles alliés transportent un identifiant optionnel de joueuse, ce qui permet d'attribuer les points au bon slot et d'éviter les collisions sémantiques entre J1 et J2.

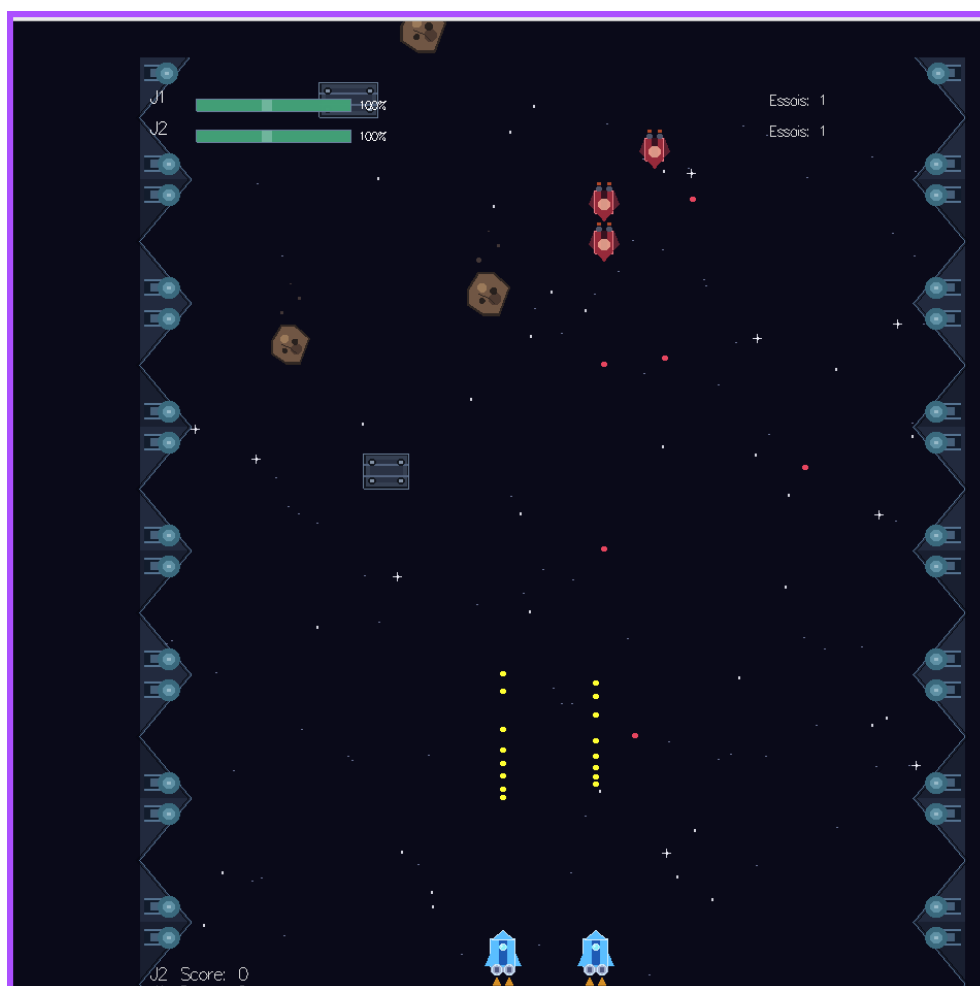


Figure 4 – Écran de jeu Multijoueur : deux joueuses, scores individuels et projectiles signés.

7.2 Génération dynamique et rejouabilité

Le module `DynamicSpawn.hs` ajoute une génération progressive d'objets. Le choix technique important est l'encadrement quantitatif : la génération enrichit la partie, mais les postconditions imposent des quotas qui empêchent l'explosion du nombre d'entités.

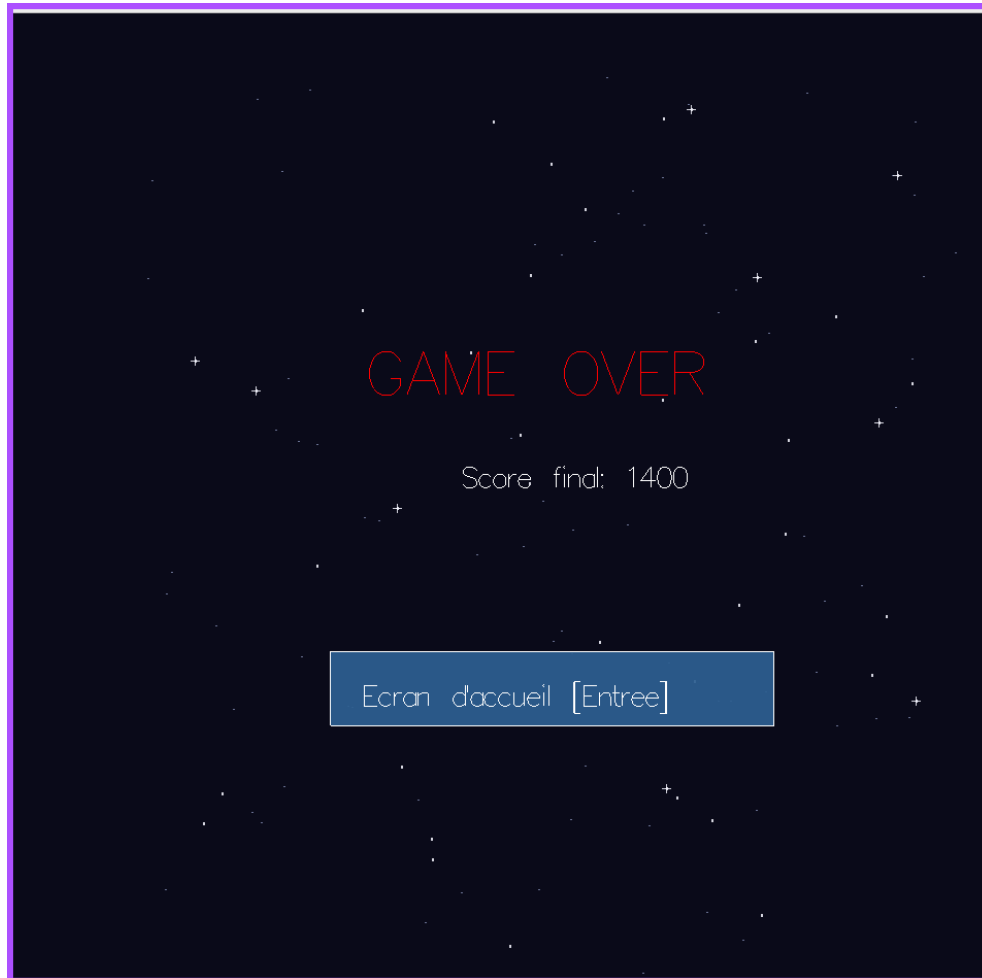


Figure 5 – Écran de Game Over : fin de l’automate de partie et retour contrôlé vers l’accueil.

7.3 Météores destructibles

Les météores créent une interaction supplémentaire entre géométrie, cadence et dégâts. Ils descendent selon leur propre rythme, peuvent sortir de l’écran et interviennent dans la résolution des collisions avec les joueuses et les ennemis. Cette extension augmente l’intérêt ludique tout en restant couverte par des invariants et des tests dédiés.

CHAPITRE 8

Synthèse quantitative de validation

Les annexes matérialisent le volume réel de vérification du projet. L'Annexe A recense **71 propriétés contractuelles** : **19 invariants**, **24 préconditions** et **28 postconditions**. L'Annexe B recense **169 tests exécutables** : **81 scénarios HSpec** et **88 propriétés QuickCheck**. Le total documentaire et exécutable atteint donc **240 éléments de validation**.

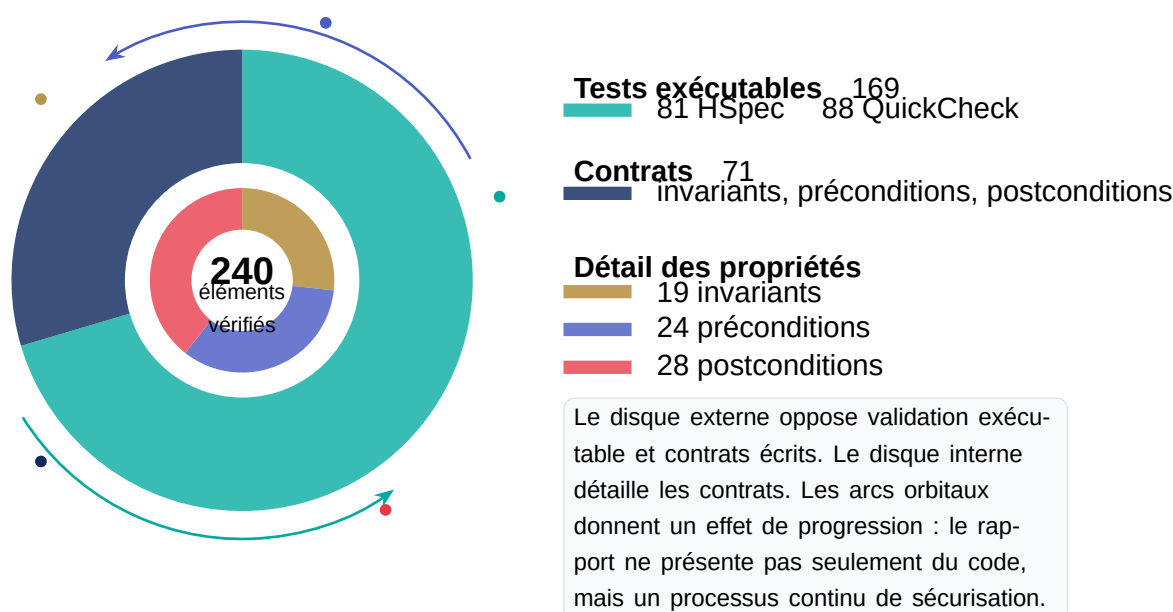


Figure 6 – Synthèse statistique des tests et propriétés documentés dans le rapport.

Cette synthèse confirme que l'effort principal n'a pas été seulement porté sur l'interface graphique. Le projet contient un volume important de garanties : les invariants vérifient la cohérence des états, les préconditions explicitent les domaines d'appel sûrs, les postconditions décrivent les effets attendus des transitions, et les tests HSpec/QuickCheck transforment ces engagements en garde de régression automatisée.

CHAPITRE 9

Conclusion

Le projet **Megablast** démontre qu'un jeu vidéo interactif peut être développé avec une discipline proche de la spécification formelle. L'usage conjoint des types, des constructeurs intelligents, des invariants et des tests de propriétés déplace la détection des erreurs vers la phase de conception. La séparation MVC rend le moteur testable sans interface graphique, tandis que les structures algébriques simplifient la composition des comportements.

Les annexes qui suivent constituent la trace exhaustive du travail de sûreté réalisé : Annexe A pour les contrats, Annexe B pour les tests et Annexe C pour les modules sources majeurs. Elles sont volontairement volumineuses, car elles documentent les garanties concrètes attendues dans un rapport PAF complet.

ANNEXE A

Répertoire exhaustif des contrats implémentés

Cette annexe recense les 71 signatures prop_inv_*, prop_pre_* et prop_post_* présentes dans les sources src/Model et src/Controller de l’archive fournie. Les contrats sont classés par module et explicitent le rôle de sûreté qu’ils jouent dans le modèle.

Table 2 – Catalogue exhaustif des contrats de sûreté extraits du code source.

#	Module	Nature	Nom	Rôle du contrat	Signature typée
1	Controller.AppController	Invariant	prop_inv_etatApplication	assure que chaque état applicatif transporte exactement les données nécessaires à son écran courant.	(a -> Bool) -> EtatApplication a -> Bool
2	Controller.AppController	Invariant	prop_inv_etatApplicationFull	spécialise l’invariant applicatif à l’état complet utilisé par le contrôleur Gloss.	EtatApplication AppStateFull -> Bool
3	Controller.Controller	Précondition	prop_pre_gererEvenement	délimite le domaine d’appel sûr de l’opération gererEvenement, en exigeant des arguments déjà compatibles avec les invariants du modèle.	AppState -> Bool
4	Controller.Controller	Postcondition	prop_post_gererEvenement	spécifie l’effet observable de gererEvenement : conservation des invariants et relation attendue entre l’état d’entrée et le résultat.	AppState -> AppState -> Bool
5	Controller.Controller	Postcondition	prop_post_appliquerCommandes	spécifie l’effet observable de appliquerCommandes : conservation des invariants et relation attendue entre l’état d’entrée et le résultat.	AppState -> Bool
6	Controller.Controller	Précondition	prop_pre_simulerStep	délimite le domaine d’appel sûr de l’opération simulerStep, en exigeant des arguments déjà compatibles avec les invariants du modèle.	AppStateFull -> Bool
7	Controller.Controller	Postcondition	prop_post_simulerStep	spécifie l’effet observable de simulerStep : conservation des invariants et relation attendue entre l’état d’entrée et le résultat.	Float -> AppStateFull -> AppStateFull -> Bool
8	Model.Engine.Collisions	Précondition	prop_pre_supprimerJoueusesEliminees	autorise la stabilisation des slots uniquement sur une liste de vaisseaux structurellement valides.	[VaisseauJoueuse] -> Bool
9	Model.Engine.Collisions	Postcondition	prop_post_supprimerJoueusesEliminees	garantit que la stabilisation ne compacte jamais la liste et conserve donc les identités J1/J2.	[VaisseauJoueuse] -> [VaisseauJoueuse] -> Bool
10	Model.Engine.Collisions	Postcondition	prop_post_reapparaitreJoueuse	formalise la transition de respawn : soit conservation si vivante, soit consommation d’un essai, soit élimination définitive.	VaisseauJoueuse -> Bool
11	Model.Engine.Commands	Précondition	prop_pre_appliquerCommande	délimite le domaine d’appel sûr de l’opération appliquerCommande, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Int -> Action -> Cadence -> Moteur -> Bool
12	Model.Engine.Commands	Postcondition	prop_post_appliquerCommande	spécifie l’effet observable de appliquerCommande : conservation des invariants et relation attendue entre l’état d’entrée et le résultat.	Int -> Action -> Cadence -> Moteur -> (Moteur, Maybe Projectile) -> Bool
13	Model.Engine.DynamicSpawn	Précondition	prop_pre_genererObjetsDynamiques	délimite le domaine d’appel sûr de l’opération genererObjetsDynamiques, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Moteur -> Bool
14	Model.Engine.DynamicSpawn	Postcondition	prop_post_genererObjetsDynamiques	spécifie l’effet observable de genererObjetsDynamiques : conservation des invariants et relation attendue entre l’état d’entrée et le résultat.	Moteur -> Moteur -> Bool
15	Model.Engine.EnnemiAI	Précondition	prop_pre_finDeTourEnnemiIntelligent	délimite le domaine d’appel sûr de l’opération finDeTourEnnemiIntelligent, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Int -> [VaisseauJoueuse] -> Ennemi -> Bool
16	Model.Engine.EnnemiCollisions	Précondition	prop_pre_resoudreCollisionsEnnemis	délimite le domaine d’appel sûr de l’opération resoudreCollisionsEnnemis, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Moteur -> Bool
17	Model.Engine.EnnemiCollisions	Postcondition	prop_post_resoudreCollisionsEnnemis	spécifie l’effet observable de resoudreCollisionsEnnemis : conservation des invariants et relation attendue entre l’état d’entrée et le résultat.	Moteur -> Moteur -> Bool
18	Model.Engine.EnnemiSpawn	Précondition	prop_pre_genererEnnemiSiBesoin	délimite le domaine d’appel sûr de l’opération genererEnnemiSiBesoin, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Moteur -> Bool

#	Module	Nature	Nom	Rôle du contrat	Signature typée
19	Model.Engine.EnnemiSpawn	Postcondition	prop_post_genererEnnemiSiBesoin	spécifie l'effet observable de genererEnnemiSiBesoin : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Moteur -> Moteur -> Bool
20	Model.Engine.GameConfig	Invariant	prop_inv_configPartie	valide que la configuration de partie décrit un mode jouable et cohérent avec l'initialisation du moteur.	ConfigPartie -> Bool
21	Model.Engine.Murs	Précondition	prop_pre_defileMurs	délimite le domaine d'appel sûr de l'opération defileMurs, en exigeant des arguments déjà compatibles avec les invariants du modèle.	MursNiveau -> Bool
22	Model.Engine.Murs	Postcondition	prop_post_defileMurs	spécifie l'effet observable de defileMurs : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	MursNiveau -> MursNiveau -> Bool
23	Model.Engine.Step	Précondition	prop_pre_finDeTourMoteur	délimite le domaine d'appel sûr de l'opération finDeTourMoteur, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Moteur -> Bool
24	Model.Engine.Step	Postcondition	prop_post_finDeTourMoteur	spécifie l'effet observable de finDeTourMoteur : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Moteur -> Moteur -> Bool
25	Model.Engine.Types	Invariant	prop_inv_evenement_planifie	interdit les événements placés sur un tour négatif dans le script moteur.	EvenementPlanifie -> Bool
26	Model.Engine.Types	Invariant	prop_inv_mursNiveau	impose la présence simultanée d'un mur gauche et d'un mur droit valides pour borner l'espace jouable.	MursNiveau -> Bool
27	Model.Engine.Types	Invariant	prop_inv_moteur	contrôle la cohérence globale de l'état de jeu : listes valides, murs, cadence, scores, tour non négatif et script trié.	Moteur -> Bool
28	Model.Hitbox	Invariant	prop_inv_hitbox	exclut les géométries dégénérées : rayon nul, rectangle d'aire nulle, mur non croissant ou composée trop courte.	Hitbox -> Bool
29	Model.Hitbox	Invariant	prop_inv_points_mur	vérifie productivement un préfixe fini d'un mur potentiellement infini, avec au moins deux points et des ordonnées croissantes.	[(Int, Int)] -> Bool
30	Model.Meteore	Invariant	prop_inv_meteore	vérifie qu'un météore possède une hitbox valide, une cadence correcte et des dégâts strictement positifs.	Meteore -> Bool
31	Model.Meteore	Précondition	prop_pre_finDeTourMeteore	délimite le domaine d'appel sûr de l'opération finDeTourMeteore, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Meteore -> Bool
32	Model.Meteore	Postcondition	prop_post_finDeTourMeteore	spécifie l'effet observable de finDeTourMeteore : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Meteore -> Meteore -> Bool
33	Model.Objects	Invariant	prop_inv_cadence	garantit une cadence temporelle productive : attente strictement positive et compteur restant borné dans l'intervalle légal.	Cadence -> Bool
34	Model.Objects	Précondition	prop_pre_tickCadence	délimite le domaine d'appel sûr de l'opération tickCadence, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Cadence -> Bool
35	Model.Objects	Postcondition	prop_post_tickCadence	spécifie l'effet observable de tickCadence : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Cadence -> (Bool, Cadence) -> Bool
36	Model.Objects	Invariant	prop_inv_obstacle	ramène la validité d'un obstacle à la validité géométrique de sa hitbox.	Obstacle -> Bool
37	Model.Objects	Invariant	prop_inv_projectile	garantit que le projectile possède une hitbox, une direction, une cadence et une signature de propriétaire compatibles.	Projectile -> Bool
38	Model.Objects	Précondition	prop_pre_avanceProjectile	délimite le domaine d'appel sûr de l'opération avanceProjectile, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Projectile -> Bool
39	Model.Objects	Postcondition	prop_post_avanceProjectile	spécifie l'effet observable de avanceProjectile : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Projectile -> Projectile -> Bool
40	Model.Objects	Précondition	prop_pre_finDeTourProjectile	délimite le domaine d'appel sûr de l'opération finDeTourProjectile, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Projectile -> Bool
41	Model.Objects	Postcondition	prop_post_finDeTourProjectile	spécifie l'effet observable de finDeTourProjectile : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Projectile -> Projectile -> Bool
42	Model.Objects	Précondition	prop_pre_finDeTourObstacles	délimite le domaine d'appel sûr de l'opération finDeTourObstacles, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Cadence -> [Obstacle] -> Bool
43	Model.Objects	Postcondition	prop_post_finDeTourObstacles	spécifie l'effet observable de finDeTourObstacles : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Cadence -> [Obstacle] -> (Cadence, [Obstacle]) -> Bool
44	Model.Objects	Invariant	prop_inv_vaisseau	contrôle la forme, les points de vie, les essais restants et la cadence de tir d'une joueuse.	VaisseauJoueuse -> Bool
45	Model.Objects	Précondition	prop_pre_essaieDeplacerVaisseau	délimite le domaine d'appel sûr de l'opération essaieDeplacerVaisseau, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Direction -> VaisseauJoueuse -> Bool
46	Model.Objects	Postcondition	prop_post_essaieDep lacerVaisseau	spécifie l'effet observable de essaieDeplacerVaisseau : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Direction -> VaisseauJoueuse -> VaisseauJoueuse -> Bool

#	Module	Nature	Nom	Rôle du contrat	Signature typée
47	Model.Objects	Précondition	prop_pre_deplaceVaisseau	délimite le domaine d'appel sûr de l'opération deplaceVaisseau, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Direction -> VaisseauJoueuse -> Bool
48	Model.Objects	Postcondition	prop_post_deplaceVaisseau	spécifie l'effet observable de deplaceVaisseau : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Direction -> VaisseauJoueuse -> VaisseauJoueuse -> Bool
49	Model.Objects	Précondition	prop_pre_repousseVaisseau	délimite le domaine d'appel sûr de l'opération repousseVaisseau, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Direction -> VaisseauJoueuse -> Bool
50	Model.Objects	Postcondition	prop_post_repousseVaisseau	spécifie l'effet observable de repousseVaisseau : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Direction -> VaisseauJoueuse -> VaisseauJoueuse -> Bool
51	Model.Objects	Précondition	prop_pre_tirVaisseau	délimite le domaine d'appel sûr de l'opération tirVaisseau, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Int -> VaisseauJoueuse -> Cadence -> Bool
52	Model.Objects	Postcondition	prop_post_tirVaisseau	spécifie l'effet observable de tirVaisseau : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Int -> VaisseauJoueuse -> Cadence -> Projectile -> Bool
53	Model.Objects	Précondition	prop_pre_subirDegat	délimite le domaine d'appel sûr de l'opération subirDegat, en exigeant des arguments déjà compatibles avec les invariants du modèle.	VaisseauJoueuse -> Bool
54	Model.Objects	Postcondition	prop_post_subirDegat	spécifie l'effet observable de subirDegat : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	VaisseauJoueuse -> VaisseauJoueuse -> Bool
55	Model.Objects	Invariant	prop_inv_oracle	vérifie que l'oracle ennemi dispose d'un comportement exploitable et ne pointe pas vers une séquence vide.	Oracle -> Bool
56	Model.Objects	Précondition	prop_pre_oracleStep	délimite le domaine d'appel sûr de l'opération oracleStep, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Oracle -> Int -> Bool
57	Model.Objects	Postcondition	prop_post_oracleStep	spécifie l'effet observable de oracleStep : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Oracle -> Int -> (Action, Oracle) -> Bool
58	Model.Objects	Invariant	prop_inv_pv	impose des points de vie strictement positifs pour les entités ennemies représentées par le type PV.	PV -> Bool
59	Model.Objects	Invariant	prop_inv_ennemi	agrège les invariants de l'ennemi : hitbox valide, points de vie strictement positifs, oracle valide et cadence de tir saine.	Ennemi -> Bool
60	Model.Objects	Précondition	prop_pre_finDeTourEnnemi	délimite le domaine d'appel sûr de l'opération finDeTourEnnemi, en exigeant des arguments déjà compatibles avec les invariants du modèle.	Int -> Ennemi -> Bool
61	Model.Objects	Postcondition	prop_post_finDeTourEnnemi	spécifie l'effet observable de finDeTourEnnemi : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Int -> Ennemi -> (Ennemi, Maybe Projectile) -> Bool
62	Model.Score	Invariant	prop_inv_score	interdit les scores négatifs afin de conserver une mesure monoïdale naturelle.	Score -> Bool
63	Model.Score	Invariant	prop_inv_scores	vérifie collectivement l'invariant de tous les scores individuels des joueuses.	[Score] -> Bool
64	Model.Score	Postcondition	prop_post_ajouterPoints	spécifie l'effet observable de ajouterPoints : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Int -> Score -> Bool
65	Model.Score	Postcondition	prop_post_ajouterPointsJoueuse	spécifie l'effet observable de ajouterPointsJoueuse : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	Int -> Int -> [Score] -> Bool
66	Model.Score	Postcondition	prop_post_scoreTotal	spécifie l'effet observable de scoreTotal : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	[Score] -> Bool
67	Model.VaisseauForme	Invariant	prop_inv_partiesVaisseau	assure que le vaisseau est composé d'un corps, de réacteurs et éventuellement d'un cockpit de formes admissibles.	PartiesVaisseau -> Bool
68	Model.VaisseauForme	Précondition	prop_pre_translatePartiesVaisseau	délimite le domaine d'appel sûr de l'opération translatePartiesVaisseau, en exigeant des arguments déjà compatibles avec les invariants du modèle.	PartiesVaisseau -> Bool
69	Model.VaisseauForme	Postcondition	prop_post_translatePartiesVaisseau	spécifie l'effet observable de translatePartiesVaisseau : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	PartiesVaisseau -> PartiesVaisseau -> Bool
70	Model.VaisseauForme	Précondition	prop_pre_hitboxPartiesVaisseau	délimite le domaine d'appel sûr de l'opération hitboxPartiesVaisseau, en exigeant des arguments déjà compatibles avec les invariants du modèle.	PartiesVaisseau -> Bool
71	Model.VaisseauForme	Postcondition	prop_post_hitboxPartiesVaisseau	spécifie l'effet observable de hitboxPartiesVaisseau : conservation des invariants et relation attendue entre l'état d'entrée et le résultat.	PartiesVaisseau -> Hitbox -> Bool

ANNEXE B

Catalogue exhaustif des tests HSpec et QuickCheck

Cette annexe recense les 169 cas de tests déclarés par les fichiers `*Spec.hs`. Les tests de type `prop` sont des propriétés QuickCheck ; les tests de type `it` sont des scénarios unitaires HSpec.

Table 3 – Catalogue exhaustif des tests extraits des fichiers de spécification.

#	Fichier	Type	Bloc	Intitulé	Logique métier vérifiée
1	AlgebraicSpec.hs :30	QuickCheck	Classes algébriques : Score	Score vérifie l’associativité de Semigroup	Dans le bloc « Classes algébriques : Score », le test vérifie que score vérifie l’associativité de Semigroup. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
2	AlgebraicSpec.hs :33	QuickCheck	Classes algébriques : Score	Score vérifie l’identité gauche de Monoid	Dans le bloc « Classes algébriques : Score », le test vérifie que score vérifie l’identité gauche de Monoid. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
3	AlgebraicSpec.hs :36	QuickCheck	Classes algébriques : Score	Score vérifie l’identité droite de Monoid	Dans le bloc « Classes algébriques : Score », le test vérifie que score vérifie l’identité droite de Monoid. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
4	AlgebraicSpec.hs :39	QuickCheck	Classes algébriques : Score	scoreTotal est mconcat des scores	Dans le bloc « Classes algébriques : Score », le test vérifie que scoreTotal est mconcat des scores. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
5	AlgebraicSpec.hs :43	QuickCheck	Classes algébriques : Score	ajouterPoints correspond à l’action du monoïde des scores pour les points positifs	Dans le bloc « Classes algébriques : Score », le test vérifie que ajouterPoints correspond à l’action du monoïde des scores pour les points positifs. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
6	AlgebraicSpec.hs :49	QuickCheck	Classes algébriques : PV	PV vérifie l’associativité de Semigroup	Dans le bloc « Classes algébriques : PV », le test vérifie que pV vérifie l’associativité de Semigroup. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
7	AlgebraicSpec.hs :52	QuickCheck	Classes algébriques : PV	la composition de deux PV valides conserve l’invariant	Dans le bloc « Classes algébriques : PV », le test vérifie que la composition de deux PV valides conserve l’invariant. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
8	AlgebraicSpec.hs :56	QuickCheck	Classes algébriques : Script	Functor : identité	Dans le bloc « Classes algébriques : Script », le test vérifie que functor : identité. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
9	AlgebraicSpec.hs :60	QuickCheck	Classes algébriques : Script	Functor : composition	Dans le bloc « Classes algébriques : Script », le test vérifie que functor : composition. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
10	AlgebraicSpec.hs :64	QuickCheck	Classes algébriques : Script	Applicative : identité	Dans le bloc « Classes algébriques : Script », le test vérifie que applicative : identité. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
11	AlgebraicSpec.hs :68	QuickCheck	Classes algébriques : Script	Applicative : homomorphisme	Dans le bloc « Classes algébriques : Script », le test vérifie que applicative : homomorphisme. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
12	AlgebraicSpec.hs :72	QuickCheck	Classes algébriques : Script	Applicative : interchange	Dans le bloc « Classes algébriques : Script », le test vérifie que applicative : interchange. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
13	AlgebraicSpec.hs :76	QuickCheck	Classes algébriques : Script	Monad : identité gauche	Dans le bloc « Classes algébriques : Script », le test vérifie que monad : identité gauche. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
14	AlgebraicSpec.hs :81	QuickCheck	Classes algébriques : Script	Monad : identité droite	Dans le bloc « Classes algébriques : Script », le test vérifie que monad : identité droite. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.

#	Fichier	Type	Bloc	Intitulé	Logique métier vérifiée
15	AlgebraicSpec.hs :85	QuickCheck	Classes algébriques : Script	Monad : associativité	Dans le bloc « Classes algébriques : Script », le test vérifie que monad : associativité. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
16	AlgebraicSpec.hs :91	QuickCheck	Classes algébriques : Script	Semigroup : associativité	Dans le bloc « Classes algébriques : Script », le test vérifie que semigroup : associativité. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
17	AlgebraicSpec.hs :97	QuickCheck	Classes algébriques : Script	Monoid : identité gauche	Dans le bloc « Classes algébriques : Script », le test vérifie que monoid : identité gauche. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
18	AlgebraicSpec.hs :101	QuickCheck	Classes algébriques : Script	Monoid : identité droite	Dans le bloc « Classes algébriques : Script », le test vérifie que monoid : identité droite. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
19	AlgebraicSpec.hs :105	QuickCheck	Classes algébriques : Script	Foldable : foldMap accumule par le monoïde choisi	Dans le bloc « Classes algébriques : Script », le test vérifie que foldable : foldMap accumule par le monoïde choisi. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
20	AlgebraicSpec.hs :109	QuickCheck	Classes algébriques : Script	Traversable : traverse inverse Script et Maybe	Dans le bloc « Classes algébriques : Script », le test vérifie que traversable : traverse inverse Script et Maybe. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
21	AlgebraicSpec.hs :113	HSpec	Classes algébriques : Script	Semigroup/Monoid compose des fragments de script moteur	Dans le bloc « Classes algébriques : Script », le test vérifie que semigroup/Monoid compose des fragments de script moteur. Il documente le comportement métier attendu et sert de garde de régression pour le module Algebraic.
22	EngineSpec.hs :36	QuickCheck	Engine.Types : MursNiveau, EvenementPlanifie et Moteur	genMoteur produit un moteur valide	Dans le bloc « Engine.Types : MursNiveau, EvenementPlanifie et Moteur », le test vérifie que genMoteur produit un moteur valide. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
23	EngineSpec.hs :39	HSpec	Engine.Types : MursNiveau, EvenementPlanifie et Moteur	mkMursNiveau exige un mur gauche et un mur droit valides	Dans le bloc « Engine.Types : MursNiveau, EvenementPlanifie et Moteur », le test vérifie que mkMursNiveau exige un mur gauche et un mur droit valides. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
24	EngineSpec.hs :45	HSpec	Engine.Types : MursNiveau, EvenementPlanifie et Moteur	mkMoteur rejette chaque famille invalide du modèle	Dans le bloc « Engine.Types : MursNiveau, EvenementPlanifie et Moteur », le test vérifie que mkMoteur rejette chaque famille invalide du modèle. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
25	EngineSpec.hs :59	HSpec	Engine.Types : MursNiveau, EvenementPlanifie et Moteur	mkMoteur trie le script par tour croissant et initialise les scores	Dans le bloc « Engine.Types : MursNiveau, EvenementPlanifie et Moteur », le test vérifie que mkMoteur trie le script par tour croissant et initialise les scores. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
26	EngineSpec.hs :70	HSpec	Engine.Murs : fenêtrage et défilement des murs	mkMursNiveau transforme les murs infinis en murs finis	Dans le bloc « Engine.Murs : fenêtrage et défilement des murs », le test vérifie que mkMursNiveau transforme les murs infinis en murs finis. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
27	EngineSpec.hs :78	QuickCheck	Engine.Murs : fenêtrage et défilement des murs	defileMurs respecte sa post-condition	Dans le bloc « Engine.Murs : fenêtrage et défilement des murs », le test vérifie que defileMurs respecte sa post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
28	EngineSpec.hs :82	HSpec	Engine.Murs : fenêtrage et défilement des murs	defileMurs garde un nombre borné de points après beaucoup de tours	Dans le bloc « Engine.Murs : fenêtrage et défilement des murs », le test vérifie que defileMurs garde un nombre borné de points après beaucoup de tours. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
29	EngineSpec.hs :90	HSpec	Engine.Step : événements et fin de tour	appliquerEvenement couvre tous les constructeurs Evenement	Dans le bloc « Engine.Step : événements et fin de tour », le test vérifie que appliquerEvenement couvre tous les constructeurs Evenement. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
30	EngineSpec.hs :113	QuickCheck	Engine.Step : événements et fin de tour	finDeTourMoteur incrémente le tour	Dans le bloc « Engine.Step : événements et fin de tour », le test vérifie que finDeTourMoteur incrémente le tour. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
31	EngineSpec.hs :116	QuickCheck	Engine.Step : événements et fin de tour	finDeTourMoteur préserve l'invariant	Dans le bloc « Engine.Step : événements et fin de tour », le test vérifie que finDeTourMoteur préserve l'invariant. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.

#	Fichier	Type	Bloc	Intitulé	Logique métier vérifiée
32	EngineSpec.hs :119	QuickCheck	Engine.Step : événements et fin de tour	les slots joueuses restent stables	Dans le bloc « Engine.Step : événements et fin de tour », le test vérifie que les slots joueuses restent stables. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
33	EngineSpec.hs :122	QuickCheck	Engine.Step : événements et fin de tour	finDeTourMoteur consomme les événements passés du script	Dans le bloc « Engine.Step : événements et fin de tour », le test vérifie que finDeTourMoteur consomme les événements passés du script. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
34	EngineSpec.hs :125	QuickCheck	Engine.Step : événements et fin de tour	finDeTourMoteur respecte sa post-condition	Dans le bloc « Engine.Step : événements et fin de tour », le test vérifie que finDeTourMoteur respecte sa post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
35	EngineSpec.hs :129	QuickCheck	Engine.Step : événements et fin de tour	finDeTourMoteurEither coïncide avec finDeTourMoteur sur un moteur valide	Dans le bloc « Engine.Step : événements et fin de tour », le test vérifie que finDeTourMoteurEither coïncide avec finDeTourMoteur sur un moteur valide. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
36	EngineSpec.hs :133	HSpec	Engine.Step : événements et fin de tour	finDeTourMoteurEither rejette un moteur invalide	Dans le bloc « Engine.Step : événements et fin de tour », le test vérifie que finDeTourMoteurEither rejette un moteur invalide. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
37	EngineSpec.hs :137	HSpec	Engine.Step : événements et fin de tour	les obstacles sortis de l'écran sont supprimés en fin de tour	Dans le bloc « Engine.Step : événements et fin de tour », le test vérifie que les obstacles sortis de l'écran sont supprimés en fin de tour. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
38	EngineSpec.hs :146	HSpec	Engine.Step : événements et fin de tour	horsEcranHitbox couvre tous les constructeurs de Hitbox	Dans le bloc « Engine.Step : événements et fin de tour », le test vérifie que horsEcranHitbox couvre tous les constructeurs de Hitbox. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
39	EngineSpec.hs :155	QuickCheck	Engine.Collisions : joueuses, projectiles, ennemis et score	supprimerJoueusesEliminees conserve les slots joueurs	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que supprimerJoueusesEliminees conserve les slots joueurs. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
40	EngineSpec.hs :161	QuickCheck	Engine.Collisions : joueuses, projectiles, ennemis et score	joueusesActives est exactement le filtre des joueuses encore en jeu	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que joueusesActives est exactement le filtre des joueuses encore en jeu. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
41	EngineSpec.hs :165	HSpec	Engine.Collisions : joueuses, projectiles, ennemis et score	reapparaitreJoueuse consomme un essai et restaure les PV si possible	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que reapparaitreJoueuse consomme un essai et restaure les PV si possible. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
42	EngineSpec.hs :172	HSpec	Engine.Collisions : joueuses, projectiles, ennemis et score	reapparaitreJoueuse élimine la joueuse au dernier essai	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que reapparaitreJoueuse élimine la joueuse au dernier essai. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
43	EngineSpec.hs :179	QuickCheck	Engine.Collisions : joueuses, projectiles, ennemis et score	reapparaitreJoueuse respecte sa post-condition	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que reapparaitreJoueuse respecte sa post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
44	EngineSpec.hs :182	HSpec	Engine.Collisions : joueuses, projectiles, ennemis et score	un projectile ennemi qui touche une joueuse disparaît et enlève 1 PV	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que un projectile ennemi qui touche une joueuse disparaît et enlève 1 PV. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
45	EngineSpec.hs :191	HSpec	Engine.Collisions : joueuses, projectiles, ennemis et score	une joueuse qui consomme son dernier essai reste dans son slot mais devient inactive	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que une joueuse qui consomme son dernier essai reste dans son slot mais devient inactive. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
46	EngineSpec.hs :204	HSpec	Engine.Collisions : joueuses, projectiles, ennemis et score	une joueuse déjà éliminée n'est pas supprimée physiquement du moteur	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que une joueuse déjà éliminée n'est pas supprimée physiquement du moteur. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
47	EngineSpec.hs :212	HSpec	Engine.Collisions : joueuses, projectiles, ennemis et score	détruire un ennemi avec un tir joueur augmente le score de la bonne joueuse	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que détruire un ennemi avec un tir joueur augmente le score de la bonne joueuse. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.

#	Fichier	Type	Bloc	Intitulé	Logique métier vérifiée
48	EngineSpec.hs :225	QuickCheck	Engine.Collisions : joueuses, projectiles, ennemis et score	resoudreCollisions préserve l'invariant du moteur	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que resoudreCollisions préserve l'invariant du moteur. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
49	EngineSpec.hs :229	HSpec	Engine.Collisions : joueuses, projectiles, ennemis et score	un contact avec le mur gauche repousse la joueuse hors du mur	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que un contact avec le mur gauche repousse la joueuse hors du mur. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
50	EngineSpec.hs :237	HSpec	Engine.Collisions : joueuses, projectiles, ennemis et score	un contact avec le mur droit repousse la joueuse hors du mur	Dans le bloc « Engine.Collisions : joueuses, projectiles, ennemis et score », le test vérifie que un contact avec le mur droit repousse la joueuse hors du mur. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
51	EngineSpec.hs :246	HSpec	Engine.Commands : commandes sûres	appliquerCommande ignore un indice de joueuse invalide	Dans le bloc « Engine.Commands : commandes sûres », le test vérifie que appliquerCommande ignore un indice de joueuse invalide. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
52	EngineSpec.hs :254	HSpec	Engine.Commands : commandes sûres	appliquerCommande ignore une joueuse inactive	Dans le bloc « Engine.Commands : commandes sûres », le test vérifie que appliquerCommande ignore une joueuse inactive. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
53	EngineSpec.hs :262	QuickCheck	Engine.Commands : commandes sûres	appliquerCommande respecte sa post-condition sur un indice valide	Dans le bloc « Engine.Commands : commandes sûres », le test vérifie que appliquerCommande respecte sa post-condition sur un indice valide. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
54	EngineSpec.hs :269	HSpec	Engine.Commands : commandes sûres	Tirer ajoute un projectile signé par la joueuse	Dans le bloc « Engine.Commands : commandes sûres », le test vérifie que tirer ajoute un projectile signé par la joueuse. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
55	EngineSpec.hs :279	HSpec	Engine.EnnemiAI	centreHitbox couvre tous les constructeurs de Hitbox	Dans le bloc « Engine.EnnemiAI », le test vérifie que centreHitbox couvre tous les constructeurs de Hitbox. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
56	EngineSpec.hs :288	HSpec	Engine.EnnemiAI	centreBasHitbox couvre tous les constructeurs de Hitbox	Dans le bloc « Engine.EnnemiAI », le test vérifie que centreBasHitbox couvre tous les constructeurs de Hitbox. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
57	EngineSpec.hs :297	HSpec	Engine.EnnemiAI	un ennemi à droite de la joueuse se déplace vers la gauche	Dans le bloc « Engine.EnnemiAI », le test vérifie que un ennemi à droite de la joueuse se déplace vers la gauche. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
58	EngineSpec.hs :307	HSpec	Engine.EnnemiAI	un ennemi à gauche de la joueuse se déplace vers la droite	Dans le bloc « Engine.EnnemiAI », le test vérifie que un ennemi à gauche de la joueuse se déplace vers la droite. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
59	EngineSpec.hs :317	HSpec	Engine.EnnemiAI	un ennemi produit un projectile ennemi quand sa cadence le permet	Dans le bloc « Engine.EnnemiAI », le test vérifie que un ennemi produit un projectile ennemi quand sa cadence le permet. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
60	EngineSpec.hs :328	QuickCheck	Engine.EnnemiAI	finDeTourEnnemiIntelligent respecte sa post-condition	Dans le bloc « Engine.EnnemiAI », le test vérifie que finDeTourEnnemiIntelligent respecte sa post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
61	EngineSpec.hs :334	HSpec	Engine.EnnemiCollisions	une collision ennemi/joueuse enlève 1 PV et pousse la joueuse vers le bas	Dans le bloc « Engine.EnnemiCollisions », le test vérifie que une collision ennemi/joueuse enlève 1 PV et pousse la joueuse vers le bas. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
62	EngineSpec.hs :346	HSpec	Engine.EnnemiCollisions	un ennemi qui touche un obstacle est repoussé vers le haut	Dans le bloc « Engine.EnnemiCollisions », le test vérifie que un ennemi qui touche un obstacle est repoussé vers le haut. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
63	EngineSpec.hs :359	HSpec	Engine.EnnemiCollisions	un ennemi qui touche un météore disparaît	Dans le bloc « Engine.EnnemiCollisions », le test vérifie que un ennemi qui touche un météore disparaît. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.

#	Fichier	Type	Bloc	Intitulé	Logique métier vérifiée
64	EngineSpec.hs :370	QuickCheck	Engine.EnnemiCollisions	resoudreCollisionsEnnemis respecte sa post-condition	Dans le bloc « Engine.EnnemiCollisions », le test vérifie que resoudreCollisionsEnnemis respecte sa post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
65	EngineSpec.hs :375	HSpec	Engine.GameConfig	la configuration par défaut crée une partie monojoueur valide	Dans le bloc « Engine.GameConfig », le test vérifie que la configuration par défaut crée une partie monojoueur valide. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
66	EngineSpec.hs :385	HSpec	Engine.GameConfig	la configuration duo crée deux joueuses et deux scores	Dans le bloc « Engine.GameConfig », le test vérifie que la configuration duo crée deux joueuses et deux scores. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
67	EngineSpec.hs :391	HSpec	Engine.GameConfig	mkJoueusesInitiales couvre MonoJoueur et DuoJoueurs	Dans le bloc « Engine.GameConfig », le test vérifie que mkJoueusesInitiales couvre MonoJoueur et DuoJoueurs. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
68	EngineSpec.hs :396	HSpec	Engine.GameConfig	définit une cadence de tir valide selon le mode	Dans le bloc « Engine.GameConfig », le test vérifie que définit une cadence de tir valide selon le mode. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
69	EngineSpec.hs :400	HSpec	Engine.GameConfig	exempleMoteur reste compatible avec la configuration par défaut	Dans le bloc « Engine.GameConfig », le test vérifie que exempleMoteur reste compatible avec la configuration par défaut. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
70	EngineSpec.hs :407	HSpec	Engine.EnnemiSpawn et DynamicSpawn	genererEnnemiSiBesoin n'ajoute pas d'ennemi hors période	Dans le bloc « Engine.EnnemiSpawn et DynamicSpawn », le test vérifie que genererEnnemiSiBesoin n'ajoute pas d'ennemi hors période. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
71	EngineSpec.hs :415	HSpec	Engine.EnnemiSpawn et DynamicSpawn	genererEnnemiSiBesoin ajoute un ennemi pendant une période d'apparition	Dans le bloc « Engine.EnnemiSpawn et DynamicSpawn », le test vérifie que genererEnnemiSiBesoin ajoute un ennemi pendant une période d'apparition. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
72	EngineSpec.hs :423	HSpec	Engine.EnnemiSpawn et DynamicSpawn	genererEnnemiSiBesoin ne dépasse pas le nombre maximal d'ennemis actifs	Dans le bloc « Engine.EnnemiSpawn et DynamicSpawn », le test vérifie que genererEnnemiSiBesoin ne dépasse pas le nombre maximal d'ennemis actifs. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
73	EngineSpec.hs :435	QuickCheck	Engine.EnnemiSpawn et DynamicSpawn	genererObjetsDynamiques respecte ses quotas et son invariant	Dans le bloc « Engine.EnnemiSpawn et DynamicSpawn », le test vérifie que genererObjetsDynamiques respecte ses quotas et son invariant. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
74	EngineSpec.hs :439	HSpec	Engine.EnnemiSpawn et DynamicSpawn	la fréquence d'apparition augmente avec la survie	Dans le bloc « Engine.EnnemiSpawn et DynamicSpawn », le test vérifie que la fréquence d'apparition augmente avec la survie. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
75	EngineSpec.hs :444	HSpec	Engine.EnnemiSpawn et DynamicSpawn	la génération dynamique ajoute au moins un objet après assez de tours	Dans le bloc « Engine.EnnemiSpawn et DynamicSpawn », le test vérifie que la génération dynamique ajoute au moins un objet après assez de tours. Il documente le comportement métier attendu et sert de garde de régression pour le module Engine.
76	HitboxSpec.hs :29	QuickCheck	Hitbox : invariants et constructeurs intelligents	tout générateur de hitbox valide satisfait prop_inv_hitbox	Dans le bloc « Hitbox : invariants et constructeurs intelligents », le test vérifie que tout générateur de hitbox valide satisfait prop_inv_hitbox. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
77	HitboxSpec.hs :32	HSpec	Hitbox : invariants et constructeurs intelligents	mkDisque rejette un rayon non strictement positif	Dans le bloc « Hitbox : invariants et constructeurs intelligents », le test vérifie que mkDisque rejette un rayon non strictement positif. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
78	HitboxSpec.hs :36	QuickCheck	Hitbox : invariants et constructeurs intelligents	mkDisque accepte exactement les rayons strictement positifs	Dans le bloc « Hitbox : invariants et constructeurs intelligents », le test vérifie que mkDisque accepte exactement les rayons strictement positifs. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
79	HitboxSpec.hs :40	HSpec	Hitbox : invariants et constructeurs intelligents	mkRectangle rejette une largeur ou hauteur non strictement positive	Dans le bloc « Hitbox : invariants et constructeurs intelligents », le test vérifie que mkRectangle rejette une largeur ou hauteur non strictement positive. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.

#	Fichier	Type	Bloc	Intitulé	Logique métier vérifiée
80	HitboxSpec.hs :45	QuickCheck	Hitbox : invariants et constructeurs intelligents	mkRectangle accepte exactement les dimensions strictement positives	Dans le bloc « Hitbox : invariants et constructeurs intelligents », le test vérifie que mkRectangle accepte exactement les dimensions strictement positives. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
81	HitboxSpec.hs :49	HSpec	Hitbox : invariants et constructeurs intelligents	mkComposee rejette les listes vides ou singletons	Dans le bloc « Hitbox : invariants et constructeurs intelligents », le test vérifie que mkComposee rejette les listes vides ou singletons. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
82	HitboxSpec.hs :53	QuickCheck	Hitbox : invariants et constructeurs intelligents	mkComposee accepte une liste de deux hitbox valides ou plus	Dans le bloc « Hitbox : invariants et constructeurs intelligents », le test vérifie que mkComposee accepte une liste de deux hitbox valides ou plus. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
83	HitboxSpec.hs :58	HSpec	Hitbox : invariants et constructeurs intelligents	l'invariant des murs refuse les listes trop courtes et les ordonnées non croissantes	Dans le bloc « Hitbox : invariants et constructeurs intelligents », le test vérifie que l'invariant des murs refuse les listes trop courtes et les ordonnées non croissantes. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
84	HitboxSpec.hs :64	HSpec	Hitbox : invariants et constructeurs intelligents	l'invariant des murs reste productif sur les murs infinis du sujet	Dans le bloc « Hitbox : invariants et constructeurs intelligents », le test vérifie que l'invariant des murs reste productif sur les murs infinis du sujet. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
85	HitboxSpec.hs :69	HSpec	Hitbox : collision de base	deux points collisionnent seulement s'ils sont superposés	Dans le bloc « Hitbox : collision de base », le test vérifie que deux points collisionnent seulement s'ils sont superposés. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
86	HitboxSpec.hs :73	HSpec	Hitbox : collision de base	Point/Rectangle : bord gauche et bas inclus, bord droit et haut exclus	Dans le bloc « Hitbox : collision de base », le test vérifie que point/Rectangle : bord gauche et bas inclus, bord droit et haut exclus. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
87	HitboxSpec.hs :80	QuickCheck	Hitbox : collision de base	Point/Rectangle est symétrique	Dans le bloc « Hitbox : collision de base », le test vérifie que point/Rectangle est symétrique. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
88	HitboxSpec.hs :86	HSpec	Hitbox : collision de base	Rectangle/Rectangle détecte les recouvrements et exclut les bords seulement tangents	Dans le bloc « Hitbox : collision de base », le test vérifie que rectangle/Rectangle détecte les recouvrements et exclut les bords seulement tangents. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
89	HitboxSpec.hs :91	QuickCheck	Hitbox : collision de base	Rectangle/Rectangle est symétrique	Dans le bloc « Hitbox : collision de base », le test vérifie que rectangle/Rectangle est symétrique. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
90	HitboxSpec.hs :98	HSpec	Hitbox : collision de base	Point/Disque inclut le centre, l'intérieur et le bord	Dans le bloc « Hitbox : collision de base », le test vérifie que point/Disque inclut le centre, l'intérieur et le bord. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
91	HitboxSpec.hs :104	QuickCheck	Hitbox : collision de base	Point/Disque est symétrique	Dans le bloc « Hitbox : collision de base », le test vérifie que point/Disque est symétrique. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
92	HitboxSpec.hs :110	HSpec	Hitbox : collision de base	Disque/Disque traite la tangence comme une collision	Dans le bloc « Hitbox : collision de base », le test vérifie que disque/Disque traite la tangence comme une collision. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
93	HitboxSpec.hs :114	QuickCheck	Hitbox : collision de base	Disque/Disque est symétrique	Dans le bloc « Hitbox : collision de base », le test vérifie que disque/Disque est symétrique. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
94	HitboxSpec.hs :120	HSpec	Hitbox : collision de base	Rectangle/Disque couvre l'intérieur, la tangence et l'absence de contact	Dans le bloc « Hitbox : collision de base », le test vérifie que rectangle/Disque couvre l'intérieur, la tangence et l'absence de contact. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
95	HitboxSpec.hs :126	QuickCheck	Hitbox : collision de base	Rectangle/Disque est symétrique	Dans le bloc « Hitbox : collision de base », le test vérifie que rectangle/Disque est symétrique. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
96	HitboxSpec.hs :134	HSpec	Hitbox : Composee	une Composee collisionne dès qu'une composante collisionne	Dans le bloc « Hitbox : Composee », le test vérifie que une Composee collisionne dès qu'une composante collisionne. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.

#	Fichier	Type	Bloc	Intitulé	Logique métier vérifiée
97	HitboxSpec.hs :140	QuickCheck	Hitbox : Composee	Q1.4 : collision avec Composee de deux points implique appartenance à ces points	Dans le bloc « Hitbox : Composee », le test vérifie que q1.4 : collision avec Composee de deux points implique appartenance à ces points. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
98	HitboxSpec.hs :144	QuickCheck	Hitbox : Composee	collision est symétrique sur les hitbox générées	Dans le bloc « Hitbox : Composee », le test vérifie que collision est symétrique sur les hitbox générées. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
99	HitboxSpec.hs :150	HSpec	Hitbox : murs	Point/MurGauche fonctionne sur le mur dent de scie de l'énoncé	Dans le bloc « Hitbox : murs », le test vérifie que point/MurGauche fonctionne sur le mur dent de scie de l'énoncé. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
100	HitboxSpec.hs :154	HSpec	Hitbox : murs	Point/MurGauche fonctionne avec un mur vertical	Dans le bloc « Hitbox : murs », le test vérifie que point/MurGauche fonctionne avec un mur vertical. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
101	HitboxSpec.hs :159	HSpec	Hitbox : murs	Point/MurDroit fonctionne avec un mur vertical	Dans le bloc « Hitbox : murs », le test vérifie que point/MurDroit fonctionne avec un mur vertical. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
102	HitboxSpec.hs :164	QuickCheck	Hitbox : murs	MurGauche/Point est symétrique	Dans le bloc « Hitbox : murs », le test vérifie que murGauche/Point est symétrique. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
103	HitboxSpec.hs :170	QuickCheck	Hitbox : murs	MurDroit/Point est symétrique	Dans le bloc « Hitbox : murs », le test vérifie que murDroit/Point est symétrique. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
104	HitboxSpec.hs :176	HSpec	Hitbox : murs	Rectangle/MurGauche collisionne si le rectangle atteint la zone du mur	Dans le bloc « Hitbox : murs », le test vérifie que rectangle/MurGauche collisionne si le rectangle atteint la zone du mur. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
105	HitboxSpec.hs :181	HSpec	Hitbox : murs	Rectangle/MurDroit collisionne si le rectangle atteint la zone du mur	Dans le bloc « Hitbox : murs », le test vérifie que rectangle/MurDroit collisionne si le rectangle atteint la zone du mur. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
106	HitboxSpec.hs :186	QuickCheck	Hitbox : murs	Rectangle/MurGauche est symétrique	Dans le bloc « Hitbox : murs », le test vérifie que rectangle/MurGauche est symétrique. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
107	HitboxSpec.hs :192	QuickCheck	Hitbox : murs	Rectangle/MurDroit est symétrique	Dans le bloc « Hitbox : murs », le test vérifie que rectangle/MurDroit est symétrique. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
108	HitboxSpec.hs :199	HSpec	Hitbox : translation exhaustive par constructeur	translateHitbox couvre Point, Disque, Rectangle, MurGauche, MurDroit et Composee	Dans le bloc « Hitbox : translation exhaustive par constructeur », le test vérifie que translateHitbox couvre Point, Disque, Rectangle, MurGauche, MurDroit et Composee. Il documente le comportement métier attendu et sert de garde de régression pour le module Hitbox.
109	MeteoreScoreSpec.hs :15	HSpec	Meteore	mkMeteore rejette hitbox, cadence ou dégâts invalides	Dans le bloc « Meteore », le test vérifie que mkMeteore rejette hitbox, cadence ou dégâts invalides. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
110	MeteoreScoreSpec.hs :21	QuickCheck	Meteore	mkMeteore accepte une hitbox, une cadence et des dégâts valides	Dans le bloc « Meteore », le test vérifie que mkMeteore accepte une hitbox, une cadence et des dégâts valides. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
111	MeteoreScoreSpec.hs :27	QuickCheck	Meteore	mkMeteoreRond produit un météore valide pour rayon et cadence valides	Dans le bloc « Meteore », le test vérifie que mkMeteoreRond produit un météore valide pour rayon et cadence valides. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
112	MeteoreScoreSpec.hs :32	QuickCheck	Meteore	avanceMeteore descend la hitbox d'un pixel et préserve l'invariant	Dans le bloc « Meteore », le test vérifie que avanceMeteore descend la hitbox d'un pixel et préserve l'invariant. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
113	MeteoreScoreSpec.hs :39	QuickCheck	Meteore	finDeTourMeteore respecte sa pré/post-condition	Dans le bloc « Meteore », le test vérifie que finDeTourMeteore respecte sa pré/post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
114	MeteoreScoreSpec.hs :44	HSpec	Meteore	un météore de cadence 1 descend à chaque tour	Dans le bloc « Meteore », le test vérifie que un météore de cadence 1 descend à chaque tour. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.

#	Fichier	Type	Bloc	Intitulé	Logique métier vérifiée
115	MeteoreScoreSpec.hs :53	QuickCheck	Score	Score généré respecte son invariant	Dans le bloc « Score », le test vérifie que score généré respecte son invariant. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
116	MeteoreScoreSpec.hs :56	HSpec	Score	scoresNuls crée exactement n scores nuls pour n positif	Dans le bloc « Score », le test vérifie que scoresNuls crée exactement n scores nuls pour n positif. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
117	MeteoreScoreSpec.hs :61	QuickCheck	Score	ajouterPoints préserve l'invariant et ignore les valeurs non positives	Dans le bloc « Score », le test vérifie que ajouterPoints préserve l'invariant et ignore les valeurs non positives. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
118	MeteoreScoreSpec.hs :68	QuickCheck	Score	ajouterPointsJoueuse préserve les scores et ne change pas la longueur	Dans le bloc « Score », le test vérifie que ajouterPointsJoueuse préserve les scores et ne change pas la longueur. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
119	MeteoreScoreSpec.hs :75	HSpec	Score	ajouterPointsJoueuse ignore les indices invalides	Dans le bloc « Score », le test vérifie que ajouterPointsJoueuse ignore les indices invalides. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
120	MeteoreScoreSpec.hs :80	HSpec	Score	ajouterPointsJoueuse crédite uniquement la joueuse ciblée	Dans le bloc « Score », le test vérifie que ajouterPointsJoueuse crédite uniquement la joueuse ciblée. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
121	MeteoreScoreSpec.hs :84	QuickCheck	Score	scoreTotal est la somme des scores individuels	Dans le bloc « Score », le test vérifie que scoreTotal est la somme des scores individuels. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
122	MeteoreScoreSpec.hs :89	HSpec	Score	scoreEnnemiDetruit est positif et augmente par paliers de survie	Dans le bloc « Score », le test vérifie que scoreEnnemiDetruit est positif et augmente par paliers de survie. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
123	MeteoreScoreSpec.hs :95	QuickCheck	Score	scoreEnnemiDetruit reste toujours au moins égal au score de base	Dans le bloc « Score », le test vérifie que scoreEnnemiDetruit reste toujours au moins égal au score de base. Il documente le comportement métier attendu et sert de garde de régression pour le module MeteoreScore.
124	ObjectsSpec.hs :13	HSpec	Direction	dirVector couvre toutes les directions du modèle	Dans le bloc « Direction », le test vérifie que dirVector couvre toutes les directions du modèle. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
125	ObjectsSpec.hs :19	QuickCheck	Direction	chaque direction déplace d'exactly un pixel sur un axe	Dans le bloc « Direction », le test vérifie que chaque direction déplace d'exactly un pixel sur un axe. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
126	ObjectsSpec.hs :23	HSpec	Cadence	mkCadence rejette attente ≤ 0	Dans le bloc « Cadence », le test vérifie que mkCadence rejette attente ≤ 0 . Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
127	ObjectsSpec.hs :27	QuickCheck	Cadence	mkCadence construit toujours une cadence valide et prête	Dans le bloc « Cadence », le test vérifie que mkCadence construit toujours une cadence valide et prête. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
128	ObjectsSpec.hs :31	QuickCheck	Cadence	tickCadence préserve l'invariant	Dans le bloc « Cadence », le test vérifie que tickCadence préserve l'invariant. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
129	ObjectsSpec.hs :35	QuickCheck	Cadence	tickCadence respecte sa post-condition	Dans le bloc « Cadence », le test vérifie que tickCadence respecte sa post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
130	ObjectsSpec.hs :39	QuickCheck	Cadence	une cadence prête bouge maintenant puis remet restant à attente-1	Dans le bloc « Cadence », le test vérifie que une cadence prête bouge maintenant puis remet restant à attente-1. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
131	ObjectsSpec.hs :43	QuickCheck	Cadence	une cadence en attente ne bouge pas et décrémente restant	Dans le bloc « Cadence », le test vérifie que une cadence en attente ne bouge pas et décrémente restant. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
132	ObjectsSpec.hs :47	HSpec	Cadence	tickCadence fait bouger exactement tous les 'attente' tours	Dans le bloc « Cadence », le test vérifie que tickCadence fait bouger exactement tous les 'attente' tours. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
133	ObjectsSpec.hs :53	QuickCheck	Obstacle	mkObstacle accepte les hitbox valides	Dans le bloc « Obstacle », le test vérifie que mkObstacle accepte les hitbox valides. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.

#	Fichier	Type	Bloc	Intitulé	Logique métier vérifiée
134	ObjectsSpec.hs :57	HSpec	Obstacle	mkObstacle rejette une hitbox invalide	Dans le bloc « Obstacle », le test vérifie que mkObstacle rejette une hitbox invalide. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
135	ObjectsSpec.hs :61	QuickCheck	Obstacle	defileObstacle préserve l'invariant	Dans le bloc « Obstacle », le test vérifie que defileObstacle préserve l'invariant. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
136	ObjectsSpec.hs :65	HSpec	Obstacle	defileObstacle descend exactement d'un pixel	Dans le bloc « Obstacle », le test vérifie que defileObstacle descend exactement d'un pixel. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
137	ObjectsSpec.hs :69	QuickCheck	Obstacle	finDeTourObstacles respecte sa pré/post-condition	Dans le bloc « Obstacle », le test vérifie que finDeTourObstacles respecte sa pré/post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
138	ObjectsSpec.hs :76	QuickCheck	Projectile	mkProjectile produit un projectile valide	Dans le bloc « Projectile », le test vérifie que mkProjectile produit un projectile valide. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
139	ObjectsSpec.hs :83	HSpec	Projectile	un projectile ennemi ne peut pas référencer une joueuse	Dans le bloc « Projectile », le test vérifie que un projectile ennemi ne peut pas référencer une joueuse. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
140	ObjectsSpec.hs :88	QuickCheck	Projectile	avanceProjectile respecte sa post-condition	Dans le bloc « Projectile », le test vérifie que avanceProjectile respecte sa post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
141	ObjectsSpec.hs :92	QuickCheck	Projectile	avanceProjectile translate la hitbox selon dirVector	Dans le bloc « Projectile », le test vérifie que avanceProjectile translate la hitbox selon dirVector. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
142	ObjectsSpec.hs :97	QuickCheck	Projectile	finDeTourProjectile préserve l'invariant et respecte la post-condition	Dans le bloc « Projectile », le test vérifie que finDeTourProjectile préserve l'invariant et respecte la post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
143	ObjectsSpec.hs :102	HSpec	Projectile	un projectile avec cadence=1 avance à chaque tour	Dans le bloc « Projectile », le test vérifie que un projectile avec cadence=1 avance à chaque tour. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
144	ObjectsSpec.hs :110	HSpec	Projectile	indiceJoueuseProjectile est défini uniquement pour les tirs de joueuse	Dans le bloc « Projectile », le test vérifie que indiceJoueuseProjectile est défini uniquement pour les tirs de joueuse. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
145	ObjectsSpec.hs :118	HSpec	Oracle et Ennemi	actionFromInt couvre les cinq branches prévues	Dans le bloc « Oracle et Ennemi », le test vérifie que actionFromInt couvre les cinq branches prévues. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
146	ObjectsSpec.hs :122	HSpec	Oracle et Ennemi	oracleStep sur Scripted est cyclique et incrémente l'indice	Dans le bloc « Oracle et Ennemi », le test vérifie que oracleStep sur Scripted est cyclique et incrémente l'indice. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
147	ObjectsSpec.hs :130	QuickCheck	Oracle et Ennemi	oracleStep respecte sa post-condition	Dans le bloc « Oracle et Ennemi », le test vérifie que oracleStep respecte sa post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
148	ObjectsSpec.hs :134	HSpec	Oracle et Ennemi	mkPV rejette les valeurs non strictement positives	Dans le bloc « Oracle et Ennemi », le test vérifie que mkPV rejette les valeurs non strictement positives. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
149	ObjectsSpec.hs :139	QuickCheck	Oracle et Ennemi	mkEnnemi produit un ennemi valide avec entrées valides	Dans le bloc « Oracle et Ennemi », le test vérifie que mkEnnemi produit un ennemi valide avec entrées valides. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
150	ObjectsSpec.hs :146	QuickCheck	Oracle et Ennemi	finDeTourEnnemi préserve l'invariant et respecte la post-condition	Dans le bloc « Oracle et Ennemi », le test vérifie que finDeTourEnnemi préserve l'invariant et respecte la post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
151	ObjectsSpec.hs :150	HSpec	Oracle et Ennemi	un ennemi Scripted Deplacer modifie sa hitbox d'un pixel	Dans le bloc « Oracle et Ennemi », le test vérifie que un ennemi Scripted Deplacer modifie sa hitbox d'un pixel. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.
152	ObjectsSpec.hs :158	HSpec	Oracle et Ennemi	un ennemi Scripted Tirer crée un projectile ennemi si sa cadence est prête	Dans le bloc « Oracle et Ennemi », le test vérifie que un ennemi Scripted Tirer crée un projectile ennemi si sa cadence est prête. Il documente le comportement métier attendu et sert de garde de régression pour le module Objects.

#	Fichier	Type	Bloc	Intitulé	Logique métier vérifiée
153	VaisseauSpec.hs :17	HSpec	PartiesVaisseau : invariants et constructeurs	mkPartiesVaisseau impose les formes attendues	Dans le bloc « PartiesVaisseau : invariants et constructeurs », le test vérifie que mkPartiesVaisseau impose les formes attendues. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
154	VaisseauSpec.hs :27	QuickCheck	PartiesVaisseau : invariants et constructeurs	mkPartiesVaisseauStandard produit toujours une forme valide	Dans le bloc « PartiesVaisseau : invariants et constructeurs », le test vérifie que mkPartiesVaisseauStandard produit toujours une forme valide. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
155	VaisseauSpec.hs :31	QuickCheck	PartiesVaisseau : invariants et constructeurs	translatePartiesVaisseau préserve l’invariant et respecte la post-condition	Dans le bloc « PartiesVaisseau : invariants et constructeurs », le test vérifie que translatePartiesVaisseau préserve l’invariant et respecte la post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
156	VaisseauSpec.hs :37	QuickCheck	PartiesVaisseau : invariants et constructeurs	hitboxPartiesVaisseau produit une hitbox composée valide	Dans le bloc « PartiesVaisseau : invariants et constructeurs », le test vérifie que hitboxPartiesVaisseau produit une hitbox composée valide. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
157	VaisseauSpec.hs :42	HSpec	PartiesVaisseau : invariants et constructeurs	pointHautCentre couvre tous les constructeurs de Hitbox	Dans le bloc « PartiesVaisseau : invariants et constructeurs », le test vérifie que pointHautCentre couvre tous les constructeurs de Hitbox. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
158	VaisseauSpec.hs :54	HSpec	VaisseauJoueuse : invariants et constructeurs	mkVaisseauJoueuse rejette pv < 0, essais < 0 ou cadence invalide	Dans le bloc « VaisseauJoueuse : invariants et constructeurs », le test vérifie que mkVaisseauJoueuse rejette pv < 0, essais < 0 ou cadence invalide. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
159	VaisseauSpec.hs :60	QuickCheck	VaisseauJoueuse : invariants et constructeurs	mkVaisseauJoueuse produit un vaisseau valide avec des entrées correctes	Dans le bloc « VaisseauJoueuse : invariants et constructeurs », le test vérifie que mkVaisseauJoueuse produit un vaisseau valide avec des entrées correctes. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
160	VaisseauSpec.hs :66	QuickCheck	VaisseauJoueuse : invariants et constructeurs	vjHitbox et vjHitboxTir produisent des hitbox valides	Dans le bloc « VaisseauJoueuse : invariants et constructeurs », le test vérifie que vjHitbox et vjHitboxTir produisent des hitbox valides. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
161	VaisseauSpec.hs :71	QuickCheck	VaisseauJoueuse : déplacements et cadence	deplaceVaisseau préserve l’invariant et respecte la post-condition	Dans le bloc « VaisseauJoueuse : déplacements et cadence », le test vérifie que deplaceVaisseau préserve l’invariant et respecte la post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
162	VaisseauSpec.hs :76	QuickCheck	VaisseauJoueuse : déplacements et cadence	repousseVaisseau annule exactement un déplacement dans la même direction	Dans le bloc « VaisseauJoueuse : déplacements et cadence », le test vérifie que repousseVaisseau annule exactement un déplacement dans la même direction. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
163	VaisseauSpec.hs :81	QuickCheck	VaisseauJoueuse : déplacements et cadence	essaieDeplacerVaisseau respecte sa post-condition	Dans le bloc « VaisseauJoueuse : déplacements et cadence », le test vérifie que essaieDeplacerVaisseau respecte sa post-condition. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
164	VaisseauSpec.hs :86	QuickCheck	VaisseauJoueuse : déplacements et cadence	si la cadence est prête, essaieDeplacerVaisseau déplace effectivement	Dans le bloc « VaisseauJoueuse : déplacements et cadence », le test vérifie que si la cadence est prête, essaieDeplacerVaisseau déplace effectivement. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
165	VaisseauSpec.hs :94	QuickCheck	VaisseauJoueuse : déplacements et cadence	si la cadence attend, essaieDeplacerVaisseau ne modifie que la cadence	Dans le bloc « VaisseauJoueuse : déplacements et cadence », le test vérifie que si la cadence attend, essaieDeplacerVaisseau ne modifie que la cadence. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
166	VaisseauSpec.hs :103	QuickCheck	VaisseauJoueuse : tir et dégâts	tirVaisseau génère un projectile valide vers le haut et signé par la joueuse	Dans le bloc « VaisseauJoueuse : tir et dégâts », le test vérifie que tirVaisseau génère un projectile valide vers le haut et signé par la joueuse. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
167	VaisseauSpec.hs :110	HSpec	VaisseauJoueuse : tir et dégâts	tirVaisseau part de la hitbox de tir du vaisseau	Dans le bloc « VaisseauJoueuse : tir et dégâts », le test vérifie que tirVaisseau part de la hitbox de tir du vaisseau. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.
168	VaisseauSpec.hs :117	QuickCheck	VaisseauJoueuse : tir et dégâts	subirDegat baisse les PV de 1 sans descendre sous 0	Dans le bloc « VaisseauJoueuse : tir et dégâts », le test vérifie que subirDegat baisse les PV de 1 sans descendre sous 0. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.

#	Fichier	Type	Bloc	Intitulé	Logique métier vérifiée
169	VaisseauSpec.hs :121	HSpec	VaisseauJoueuse : tir et dégâts	subirDegat bloque les PV à 0	Dans le bloc « VaisseauJoueuse : tir et dégâts », le test vérifie que subirDegat bloque les PV à 0. Il documente le comportement métier attendu et sert de garde de régression pour le module Vaisseau.

ANNEXE C

Code source majeur

Cette annexe insère les trois modules demandés sous forme de `lstlisting` complets : générateurs QuickCheck, boucle de simulation et résolution des collisions avec slots stables.

C.1 SpecHelpers.hs

```

1 {-# OPTIONS_GHC -Wno-orphans #-}
2
3 module SpecHelpers where
4
5 import Data.Text (Text)
6 import Model.Engine
7 import Model.Hitbox
8 import Model.Meteore
9 import Model.Objects
10 import Model.Score
11 import Model.VaisseauForme
12 import Test.QuickCheck
13
14 newtype SmallInt = SmallInt Int deriving (Show)
15
16 instance Arbitrary SmallInt where
17   arbitrary = SmallInt <$> chooseInt (-100, 100)
18
19 isLeft :: Either a b -> Bool
20 isLeft (Left _) = True
21 isLeft _       = False
22
23 isRight :: Either a b -> Bool
24 isRight (Right _) = True
25 isRight _        = False
26
27 isRightWith :: (b -> Bool) -> Either a b -> Bool
28 isRightWith pr (Right x) = pr x
29 isRightWith _ _         = False
30
31 unsafeRight :: String -> Either e a -> a
32 unsafeRight _ (Right x) = x
33 unsafeRight msg (Left _) = error msg
34
35 instance Arbitrary Direction where
36   arbitrary = elements [minBound .. maxBound]
37
38 instance Arbitrary ProjectileOwner where
39   arbitrary = elements [minBound .. maxBound]
40
41 instance Arbitrary Action where
42   arbitrary = oneof
43     [ pure Attendre
44     , pure Tirer
45     , Deplacer <$> arbitrary
46     ]
47
48 smallPositive :: Gen Int
49 smallPositive = chooseInt (1, 40)
50

```

C. Code source majeur

```

51 genPointCoords :: Gen (Int, Int)
52 genPointCoords =
53   (,) <$> chooseInt (-200, 1000) <*> chooseInt (-200, 1100)
54
55 genPointHitbox :: Gen Hitbox
56 genPointHitbox = uncurry Point <$> genPointCoords
57
58 genDisqueHitbox :: Gen Hitbox
59 genDisqueHitbox = do
60   (x, y) <- genPointCoords
61   r <- chooseInt (1, 40)
62   pure (Disque x y r)
63
64 genRectHitbox :: Gen Hitbox
65 genRectHitbox = do
66   x <- chooseInt (-100, largeurZoneJeu + 100)
67   y <- chooseInt (-100, hauteurZoneJeu + 100)
68   w <- chooseInt (1, 80)
69   h <- chooseInt (1, 80)
70   pure (Rectangle x y w h)
71
72 genMurGaucheFini :: Gen Hitbox
73 genMurGaucheFini = do
74   x0 <- chooseInt (0, 80)
75   y0 <- chooseInt (-100, 50)
76   dy <- chooseInt (1, 80)
77   n <- chooseInt (2, 10)
78   pure (MurGauche [(x0 + (i `mod` 2) * 15, y0 + i * dy) | i <- [0 .. n - 1]])
79
80 genMurDroitFini :: Gen Hitbox
81 genMurDroitFini = do
82   x0 <- chooseInt (largeurZoneJeu - 80, largeurZoneJeu)
83   y0 <- chooseInt (-100, 50)
84   dy <- chooseInt (1, 80)
85   n <- chooseInt (2, 10)
86   pure (MurDroit [(x0 - (i `mod` 2) * 15, y0 + i * dy) | i <- [0 .. n - 1]])
87
88 genMobileHitbox :: Gen Hitbox
89 genMobileHitbox = sized go
90   where
91     go n
92       | n <= 0 = oneof [genPointHitbox, genDisqueHitbox, genRectHitbox]
93       | otherwise = frequency
94         [ (5, genPointHitbox)
95         , (5, genDisqueHitbox)
96         , (7, genRectHitbox)
97         , (2, do
98             k <- chooseInt (2, 4)
99             hs <- vectorOf k (go (n `div` 2))
100             pure (Composee hs))
101         ]
102
103 genHitbox :: Int -> Gen Hitbox
104 genHitbox n
105   | n <= 0 = oneof [genPointHitbox, genDisqueHitbox, genRectHitbox]
106   | otherwise = frequency
107     [ (5, genPointHitbox)
108     , (5, genDisqueHitbox)
109     , (7, genRectHitbox)
110     , (2, do
111         k <- chooseInt (2, 4)
112         hs <- vectorOf k (genHitbox (n `div` 2))

```

C. Code source majeur

```

113     pure (Composee hs))
114     , (1, genMurGaucheFinis)
115     , (1, genMurDroitFinis)
116   ]
117
118 instance Arbitrary Hitbox where
119   arbitrary = sized genHitbox
120
121 instance Arbitrary Cadence where
122   arbitrary = genCadence
123
124 genCadence :: Gen Cadence
125 genCadence = do
126   a <- chooseInt (1, 20)
127   r <- chooseInt (0, a - 1)
128   pure (Cadence a r)
129
130 genCadencePrete :: Gen Cadence
131 genCadencePrete = Cadence <$> chooseInt (1, 20) <*> pure 0
132
133 genCadenceEnAttente :: Gen Cadence
134 genCadenceEnAttente = do
135   a <- chooseInt (2, 20)
136   r <- chooseInt (1, a - 1)
137   pure (Cadence a r)
138
139 instance Arbitrary PartiesVaisseau where
140   arbitrary = genPartiesVaisseau
141
142 genPartiesVaisseau :: Gen PartiesVaisseau
143 genPartiesVaisseau = do
144   x <- chooseInt (50, largeurZoneJeu - 100)
145   y <- chooseInt (20, hauteurZoneJeu - 100)
146   pure $ unsafeRight
147     "genPartiesVaisseau: mkPartiesVaisseauStandard impossible"
148     (mkPartiesVaisseauStandard x y)
149
150 instance Arbitrary Projectile where
151   arbitrary = genProjectile
152
153 genProjectile :: Gen Projectile
154 genProjectile = do
155   h <- genMobileHitbox
156   d <- arbitrary
157   c <- genCadence
158   owner <- arbitrary
159   joueur <- case owner of
160     TirJoueuse -> frequency
161       [ (1, pure Nothing)
162       , (4, Just <$> chooseInt (0, 3))
163       ]
164     TirEnnemi -> pure Nothing
165   pure (Projectile h d c owner joueur)
166
167 genProjectileJoueuse :: Gen Projectile
168 genProjectileJoueuse = do
169   h <- genMobileHitbox
170   d <- arbitrary
171   c <- genCadence
172   i <- chooseInt (0, 3)
173   pure (Projectile h d c TirJoueuse (Just i))
174

```

C. Code source majeur

```

175 genProjectileEnnemi :: Gen Projectile
176 genProjectileEnnemi = do
177   h <- genMobileHitbox
178   d <- arbitrary
179   c <- genCadence
180   pure (Projectile h d c TirEnnemi Nothing)
181
182 instance Arbitrary Obstacle where
183   arbitrary = genObstacleSimple
184
185 genObstacleSimple :: Gen Obstacle
186 genObstacleSimple = Obstacle <$> genRectHitbox
187
188 instance Arbitrary VaisseauJoueuse where
189   arbitrary = genVaisseau
190
191 genVaisseau :: Gen VaisseauJoueuse
192 genVaisseau = do
193   forme <- genPartiesVaisseau
194   pv <- chooseInt (0, pvMaxJoueuse)
195   essais <- chooseInt (0, 5)
196   cad <- genCadence
197   pure (VaisseauJoueuse forme pv essais cad)
198
199 genVaisseauActif :: Gen VaisseauJoueuse
200 genVaisseauActif = do
201   forme <- genPartiesVaisseau
202   pv <- chooseInt (1, pvMaxJoueuse)
203   essais <- chooseInt (1, 5)
204   cad <- genCadence
205   pure (VaisseauJoueuse forme pv essais cad)
206
207 genVaisseauElimine :: Gen VaisseauJoueuse
208 genVaisseauElimine = do
209   forme <- genPartiesVaisseau
210   cad <- genCadence
211   oneof
212     [ pure (VaisseauJoueuse forme 0 0 cad)
213     , do pv <- chooseInt (0, pvMaxJoueuse)
214       pure (VaisseauJoueuse forme pv 0 cad)
215     ]
216
217 instance Arbitrary Oracle where
218   arbitrary = genOracle
219
220 genOracle :: Gen Oracle
221 genOracle = frequency
222   [ (4, do
223     k <- chooseInt (1, 8)
224     actions <- vectorOf k arbitrary
225     i <- chooseInt (0, 20)
226     pure (Scripted actions i))
227   , (1, pure FromInt)
228   ]
229
230 instance Arbitrary PV where
231   arbitrary = PV <$> chooseInt (1, 20)
232
233 instance Arbitrary Ennemi where
234   arbitrary = genEnnemiSimple
235
236 genEnnemi :: Gen Ennemi

```

C. Code source majeur

```

237 genEnnemi = genEnnemiSimple
238
239 genEnnemiSimple :: Gen Ennemi
240 genEnnemiSimple = do
241   h <- genRectHitbox
242   pv <- chooseInt (1, 8)
243   cad <- genCadence
244   o <- genOracle
245   pure (Ennemi h (PV pv) o cad)
246
247 instance Arbitrary Meteore where
248   arbitrary = genMeteoreSimple
249
250 genMeteoreSimple :: Gen Meteore
251 genMeteoreSimple = do
252   x <- chooseInt (50, largeurZoneJeu - 50)
253   y <- chooseInt (-50, hauteurZoneJeu + 100)
254   r <- chooseInt (1, 30)
255   cad <- genCadence
256   degats <- chooseInt (1, 5)
257   pure (Meteore (Disque x y r) cad degats)
258
259 instance Arbitrary Score where
260   arbitrary = Score <$> chooseInt (0, 100000)
261
262 genScores :: Gen [Score]
263 genScores = listOf arbitrary
264
265 instance Arbitrary Evenement where
266   arbitrary = oneof
267     [ AppEnnemi <$> genEnnemiSimple
268     , AppObstacle <$> genObstacleSimple
269     , AppProjectile <$> genProjectile
270     , AppMeteore <$> genMeteoreSimple
271     , DisparEnnemi <$> chooseInt (-2, 8)
272     , DisparObstacle <$> chooseInt (-2, 8)
273     , DisparMeteore <$> chooseInt (-2, 8)
274     ]
275
276 instance Arbitrary EvenementPlanifie where
277   arbitrary = genEvenementPlanifieSimple
278
279 genEvenementPlanifieSimple :: Gen EvenementPlanifie
280 genEvenementPlanifieSimple = do
281   tour <- chooseInt (0, 30)
282   evt <- arbitrary
283   pure (EvenementPlanifie tour evt)
284
285 mursTest :: MursNiveau
286 mursTest = unsafeRight "mursTest invalide" $
287   mkMursNiveau
288     (MurGauche [(0, 0), (0, hauteurZoneJeu + 200)])
289     (MurDroit [(largeurZoneJeu, 0), (largeurZoneJeu, hauteurZoneJeu + 200)])
290
291 mkMoteurTest
292   :: [Obstacle]
293   -> [Projectile]
294   -> [Ennemi]
295   -> [VaisseauJoueuse]
296   -> Cadence
297   -> [EvenementPlanifie]
298   -> Int

```

C. Code source majeur

```

299   -> Int
300   -> Either Text Moteur
301 mkMoteurTest obs projs enns jous cad evts tour seed =
302   mkMoteur obs projs enns jous mursTest cad evts tour seed
303
304 genMoteur :: Gen Moteur
305 genMoteur = do
306   nbObs <- chooseInt (0, 4)
307   nbProj <- chooseInt (0, 5)
308   nbEnn <- chooseInt (0, 4)
309   nbMet <- chooseInt (0, 2)
310   nbJ <- chooseInt (1, 2)
311
312   obs <- vectorOf nbObs genObstacleSimple
313   projs <- vectorOf nbProj genProjectile
314   enns <- vectorOf nbEnn genEnnemiSimple
315   meteores <- vectorOf nbMet genMeteoreSimple
316   jous <- vectorOf nbJ genVaisseauActif
317
318   cad <- genCadence
319   evts <- listOf genEvenementPlanifieSimple
320   tour <- chooseInt (0, 30)
321   seed <- arbitrary
322
323   case mkMoteurTest obs projs enns jous cad evts tour seed of
324     Right m -> pure (m { mMeteores = meteores })
325     Left _   -> genMoteur
326
327 instance Arbitrary Moteur where
328   arbitrary = genMoteur
329
330 moteurMinimal :: Moteur
331 moteurMinimal =
332   let cad = unsafeRight "cadence minimale invalide" (mkCadence 1)
333       forme = unsafeRight "forme minimale invalide" (mkPartiesVaisseauStandard 100 100)
334       v = unsafeRight "vaisseau minimal invalide" (mkVaisseauJoueuse forme 3 2 cad)
335   in unsafeRight "moteur minimal invalide" (mkMoteurTest [] [] [] [v] cad [] 0 0)

```

Listing 1 – Module SpecHelpers.hs : générateurs QuickCheck constructifs sans suchThat

C.2 Step.hs

```

1 {-# LANGUAGE OverloadedStrings #-}
2
3 module Model.Engine.Step where
4
5 import Data.Text (Text)
6 import Model.Engine.Collisions
7 import Model.Engine.EnnemiSpawn
8 import Model.Engine.EnnemiAI
9 import Model.Engine.EnnemiCollisions
10 import Model.Engine.ListUtils
11 import Model.Engine.Murs
12 import Model.Engine.Types
13 import Model.Hitbox
14 import Model.Meteore
15 import Model.Objects
16 import System.Random (StdGen, random)
17 import Model.Engine.DynamicSpawn
18

```


C. Code source majeur

```

19 -- | Application d'un événement du script.
20 appliquerEvenement :: Evenement -> Moteur -> Moteur
21 appliquerEvenement (AppEnnemi e)      m = m { mEnnemis      = mEnnemis m ++ [e] }
22 appliquerEvenement (AppObstacle o)    m = m { mObstacles    = mObstacles m ++ [o] }
23 appliquerEvenement (AppProjectile p)   m = m { mProjectiles   = mProjectiles m ++ [p] }
24 appliquerEvenement (AppMeteore mt)    m = m { mMeteores     = mMeteores m ++ [mt] }
25 appliquerEvenement (DisparEnnemi i)    m = m { mEnnemis      = removeAt i (mEnnemis m) }
26 appliquerEvenement (DisparObstacle i) m = m { mObstacles    = removeAt i (mObstacles m) }
27 appliquerEvenement (DisparMeteore i)   m = m { mMeteores     = removeAt i (mMeteores m) }
28
29 prop_pre_finDeTourMoteur :: Moteur -> Bool
30 prop_pre_finDeTourMoteur m =
31   prop_inv_moteur m && prop_partie_en_cours m
32
33 prop_post_finDeTourMoteur :: Moteur -> Moteur -> Bool
34 prop_post_finDeTourMoteur m m' =
35   prop_inv_moteur m'
36   && mTour m' == mTour m + 1
37   && length (mJoueuses m') == length (mJoueuses m)
38   && all prop_inv_projectile (mProjectiles m')
39
40 -- | Version sûre de la fin de tour. Elle rend explicite l'échec par `Either`
41 --   au lieu de supposer implicitement que l'invariant est déjà respecté.
42 finDeTourMoteurEither :: Moteur -> Either Text Moteur
43 finDeTourMoteurEither m
44   | not (prop_inv_moteur m) = Left "finDeTourMoteur: invariant moteur violé"
45   | otherwise =
46     let tourActuel =
47       mTour m
48
49     (evtsNow, evtsFutur) =
50       partitionScript (\ep -> epTour ep <= tourActuel) (mScript m)
51
52     m0 =
53       foldr (appliquerEvenement . epEvenement) m evtsNow
54
55     m1 =
56       m0 { mScript = evtsFutur }
57
58     -- Nouvelle étape : apparition automatique des ennemis.
59     mSpawn =
60       genererObjetsDynamiques m1
61
62     (scrollNow, cadScroll') =
63       tickCadence (mCadScroll mSpawn)
64
65     obsDefiles =
66       if scrollNow
67       then map defileObstacle (mObstacles mSpawn)
68       else mObstacles mSpawn
69
70     obsValides =
71       filter (not . horsEcranMoteur . obsHitbox) obsDefiles
72
73     murs' =
74       if scrollNow
75       then defileMurs (mMurs mSpawn)
76       else mMurs mSpawn
77
78     meteoresTour =
79       map finDeTourMeteore (mMeteores mSpawn)
80

```

C. Code source majeur

```

81     meteoeresValides =
82         filter (not . horsEcranMoteur . mtHitbox) meteoeresTour
83
84     projs' =
85         map finDeTourProjectile (mProjectiles mSpawn)
86
87     rng =
88         mRng mSpawn
89
90     (graineEnnemis, rngNext) =
91         random rng :: (Int, StdGen)
92
93     (ennsTour, newProjs) =
94         unzip $
95             map
96                 (finDeTourEnnemiIntelligent graineEnnemis (mJoueuses mSpawn))
97                 (mEnnemis mSpawn)
98
99     ennsValides =
100         filter (not . horsEcranMoteur . eHitbox) ennsTour
101
102     projsEnnemis =
103         [ p | Just p <- newProjs ]
104
105     tousProjs =
106         projs' ++ projsEnnemis
107
108     projsValides =
109         filter (not . horsEcranMoteur . prHitbox) tousProjs
110
111     m2 =
112         resoudreCollisionsEnnemis $
113             resoudreCollisions
114                 (mSpawn { mObstacles = obsValides
115                           , mProjectiles = projsValides
116                           , mEnnemis = ennsValides
117                           , mMeteores = meteoeresValides
118                           , mMurs = murs'
119                           , mCadScroll = cadScroll'
120                           })
121
122     m3 =
123         m2 { mTour = tourActuel + 1
124             , mRng = rngNext
125             }
126
127     in Right m3
128
129 finDeTourMoteur :: Moteur -> Moteur
130 finDeTourMoteur m =
131     case finDeTourMoteurEither m of
132         Right m' -> m'
133         Left _ -> m
134
135 -- | Détermine si une hitbox est hors de l'écran logique du moteur.
136 horsEcranHitbox :: Int -> Int -> Int -> Hitbox -> Bool
137 horsEcranHitbox largeur hauteur marge (Point x y) =
138     x < (-marge) || x > largeur + marge || y < (-marge) || y > hauteur + marge
139
140 horsEcranHitbox largeur hauteur marge (Disque xc yc r) =
141     xc + r < (-marge) || xc - r > largeur + marge
142     || yc + r < (-marge) || yc - r > hauteur + marge

```

C. Code source majeur

```

143
144 horsEcranHitbox largeur hauteur marge (Model.Hitbox.Rectangle x y w h) =
145   x + w < (-marge) || x > largeur + marge
146   || y + h < (-marge) || y > hauteur + marge
147
148 horsEcranHitbox largeur hauteur marge (Composee hs) =
149   all (horsEcranHitbox largeur hauteur marge) hs
150
151 horsEcranHitbox _ _ _ _ =
152   False
153
154 horsEcranMoteur :: Hitbox -> Bool
155 horsEcranMoteur =
156   horsEcranHitbox largeurZoneJeu hauteurZoneJeu margeHorsEcran

```

Listing 2 – Module Step.hs : boucle de simulation pure du moteur

C.3 Collisions.hs

```

1 {-# LANGUAGE OverloadedStrings #-}
2
3 module Model.Engine.Collisions where
4
5 import Model.Engine.Types
6 import Model.Hitbox
7 import Model.Meteore
8 import Model.Objects
9 import Model.Score
10
11 -- | Une joueuse est encore jouable seulement s'il lui reste au moins un essai
12 --   et au moins un point de vie.
13 --
14 -- Convention :
15 --   - essais > 0 : la joueuse peut encore jouer ;
16 --   - essais == 0 : la joueuse est éliminée.
17 --
18 -- Important en multijoueur : on ne doit jamais compacter la liste
19 --   mJoueuses après une élimination. Les indices de cette liste sont les
20 --   identités stables des joueuses : indice 0 = J1, indice 1 = J2.
21 --   Si J1 disparaît physiquement de la liste, J2 prend l'indice 0 et récupère
22 --   les touches ainsi que le score de J1.
23 joueuseEncoreEnJeu :: VaisseauJoueuse -> Bool
24 joueuseEncoreEnJeu v =
25   vjPv v > 0 && vjEssais v > 0
26
27 joueusesActives :: [VaisseauJoueuse] -> [VaisseauJoueuse]
28 joueusesActives =
29   filter joueuseEncoreEnJeu
30
31 -- | Conservé pour compatibilité avec le reste du moteur, mais cette fonction
32 --   ne supprime plus les joueuses éliminées. Elle stabilise les slots joueurs.
33 --   La disparition visuelle se fait côté vue avec joueusesActives.
34 supprimerJoueusesEliminees :: [VaisseauJoueuse] -> [VaisseauJoueuse]
35 supprimerJoueusesEliminees =
36   id
37
38 prop_pre_supprimerJoueusesEliminees :: [VaisseauJoueuse] -> Bool
39 prop_pre_supprimerJoueusesEliminees =
40   all prop_inv_vaisseau
41

```

C. Code source majeur

```

42 prop_post_supprimerJoueusesEliminees :: [VaisseauJoueuse] -> [VaisseauJoueuse] -> Bool
43 prop_post_supprimerJoueusesEliminees js js' =
44     all prop_inv_vaisseau js'
45     && length js' == length js
46
47 prop_partie_en_cours :: Moteur -> Bool
48 prop_partie_en_cours m =
49     any joueuseEncoreEnJeu (mJoueuses m)
50
51 pvRespawnJoueuse :: Int
52 pvRespawnJoueuse =
53     pvMaxJoueuse
54
55 distanceRepousseMur :: Int
56 distanceRepousseMur = 10
57
58 limiteCorrectionMur :: Int
59 limiteCorrectionMur = 80
60
61 repousseVaisseauN :: Int -> Direction -> VaisseauJoueuse -> VaisseauJoueuse
62 repousseVaisseauN n d v
63   | n <= 0    = v
64   | otherwise = repousseVaisseauN (n - 1) d (repousseVaisseau d v)
65
66 repousseHorsMur :: Hitbox -> Direction -> VaisseauJoueuse -> VaisseauJoueuse
67 repousseHorsMur mur d =
68     ajouterImpulsion . corriger limiteCorrectionMur
69     where
70         corriger 0 v = v
71         corriger n v
72           | collision (vjHitbox v) mur = corriger (n - 1) (repousseVaisseau d v)
73           | otherwise                 = v
74
75         ajouterImpulsion v =
76             repousseVaisseauN distanceRepousseMur d v
77
78 reapparaitreJoueuse :: VaisseauJoueuse -> VaisseauJoueuse
79 reapparaitreJoueuse v
80   | vjPv v > 0 =
81       v
82
83   | vjEssais v <= 0 =
84       v { vjPv = 0, vjEssais = 0 }
85
86   | vjEssais v == 1 =
87       v { vjPv = 0, vjEssais = 0 }
88
89   | otherwise =
90       v { vjPv = pvRespawnJoueuse, vjEssais = vjEssais v - 1 }
91
92 prop_post_reapparaitreJoueuse :: VaisseauJoueuse -> Bool
93 prop_post_reapparaitreJoueuse v =
94     let v' = reapparaitreJoueuse v
95     in  prop_inv_vaisseau v'
96         && case () of
97             _ | vjPv v > 0 ->
98                 v' == v
99
100             | vjEssais v <= 1 ->
101                 vjPv v' == 0
102                 && vjEssais v' == 0
103

```

C. Code source majeur

```

104         | otherwise ->
105             vjPv v' == pvRespawnJoueuse
106             && vjEssais v' == vjEssais v - 1
107
108 -- | Applique un dégât à une joueuse puis gère immédiatement la transition vers
109 -- l'état "essai consommé / respawn" quand c'est nécessaire.
110 encaisserDegatJoueuse :: VaisseauJoueuse -> VaisseauJoueuse
111 encaisserDegatJoueuse = reapparaîtreJoueuse . subirDegat
112
113 encaisserDegatsJoueuse :: Int -> VaisseauJoueuse -> VaisseauJoueuse
114 encaisserDegatsJoueuse n v
115     | n <= 0     = v
116     | otherwise = encaisserDegatsJoueuse (n - 1) (encaisserDegatJoueuse v)
117
118 joueuseToucheePar :: VaisseauJoueuse -> Projectile -> (VaisseauJoueuse, Bool)
119 joueuseToucheePar v p
120     | not (joueuseEncoreEnJeu v) =
121         (v, False)
122     | prOwner p == TirEnnemi
123       && collision (vjHitbox v) (prHitbox p) = (encaisserDegatJoueuse v, True)
124     | otherwise = (v, False)
125
126 joueuseToucheeMeteore :: VaisseauJoueuse -> Meteoire -> (VaisseauJoueuse, Bool)
127 joueuseToucheeMeteore v mt
128     | not (joueuseEncoreEnJeu v) =
129         (v, False)
130     | collision (vjHitbox v) (mtHitbox mt) =
131         (encaisserDegatsJoueuse (mtDegats mt) v, True)
132     | otherwise =
133         (v, False)
134
135 ennemiToucheePar :: Ennemi -> Projectile -> (Maybe Ennemi, Bool)
136 ennemiToucheePar e p =
137     let (me, touche, _) = ennemiToucheeParAvecMort e p
138     in (me, touche)
139
140 ennemiToucheeParAvecMort :: Ennemi -> Projectile -> (Maybe Ennemi, Bool, Bool)
141 ennemiToucheeParAvecMort e p
142     | estTirJoueuse p
143       && collision (eHitbox e) (prHitbox p) =
144         let (PV pv) = ePV e
145             pv' = pv - 1
146             mort = pv' <= 0
147         in (if mort then Nothing else Just (e { ePV = PV pv' })), True, mort)
148
149     | otherwise =
150         (Just e, False, False)
151
152 joueuseToucheeObstacle :: VaisseauJoueuse -> Obstacle -> VaisseauJoueuse
153 joueuseToucheeObstacle v o
154     | not (joueuseEncoreEnJeu v) = v
155     | collision (vjHitbox v) (obsHitbox o) = repousseVaisseau Haut v
156     | otherwise = v
157
158 joueuseToucheeEnnemi :: VaisseauJoueuse -> Ennemi -> VaisseauJoueuse
159 joueuseToucheeEnnemi v e
160     | not (joueuseEncoreEnJeu v) = v
161     | collision (vjHitbox v) (eHitbox e) = repousseVaisseau Haut (encaisserDegatJoueuse v)
162     | otherwise = v
163
164 joueuseToucheeMurGauche :: Hitbox -> VaisseauJoueuse -> VaisseauJoueuse
165 joueuseToucheeMurGauche mur v

```

C. Code source majeur

```

166 | not (joueuseEncoreEnJeu v) = v
167 | collision (vjHitbox v) mur = repousseHorsMur mur Gauche v
168 | otherwise                = v
169
170 joueuseToucheeMurDroit :: Hitbox -> VaisseauJoueuse -> VaisseauJoueuse
171 joueuseToucheeMurDroit mur v
172 | not (joueuseEncoreEnJeu v) = v
173 | collision (vjHitbox v) mur = repousseHorsMur mur Droite v
174 | otherwise                = v
175
176 joueuseToucheeMurs :: MursNiveau -> VaisseauJoueuse -> VaisseauJoueuse
177 joueuseToucheeMurs murs =
178   joueuseToucheeMurDroit (mnMurDroit murs)
179   . joueuseToucheeMurGauche (mnMurGauche murs)
180
181 resoudreCollisions :: Moteur -> Moteur
182 resoudreCollisions m =
183   let
184     joueusesAuDepart =
185       supprimerJoueusesEliminees (mJoueuses m)
186
187     (jous1, projsApresJoueuses) =
188       foldr appliquerProjSurJoueuses (joueusesAuDepart, []) (mProjectiles m)
189
190     appliquerProjSurJoueuses p (js, acc) =
191       let (js', consomme) = parcourirJoueuses p js
192       in if consomme
193         then (js', acc)
194         else (js', p : acc)
195
196     parcourirJoueuses _ [] =
197       ([], False)
198
199     parcourirJoueuses p (j:js) =
200       let (j', hit) = joueuseToucheePar j p
201       in if hit
202         then (supprimerJoueusesEliminees (j' : js), True)
203         else
204           let (js', found) = parcourirJoueuses p js
205           in (j : js', found)
206
207     m1 =
208       m { mJoueuses = joues1
209         , mProjectiles = projsApresJoueuses
210         }
211
212     (enns2, projsRestes, scoresGagnes) =
213       foldr appliquerProjSurEnnemis (mEnnemis m1, [], mScores m1) (mProjectiles m1)
214
215     appliquerProjSurEnnemis p (es, acc, scoresAcc) =
216       let (es', consomme, ennemiDetruit) = parcourirEnnemis p es
217       in scoresAcc' =
218         if ennemiDetruit
219         then crediterScoreProjectile p (scoreEnnemiDetruit (mTour m1)) scoresAcc
220         else scoresAcc
221
222       in if consomme
223         then (es', acc, scoresAcc')
224         else (es', p : acc, scoresAcc')
225
226     crediterScoreProjectile p points scores =
227       case indiceJoueuseProjectile p of
228         Nothing ->

```

C. Code source majeur

```

228     scores
229
230     Just indiceJoueuse ->
231     ajouterPointsJoueuse indiceJoueuse points scores
232
233     parcourirEnnemis _ [] =
234     ([], False, False)
235
236     parcourirEnnemis p (e:es) =
237     let (me', hit, mort) = ennemiToucheeParAvecMort e p
238     es' = maybe es (:es) me'
239     in if hit
240     then (es', True, mort)
241     else
242     let (es'', found, mortTrouvee) = parcourirEnnemis p es
243     in (e : es'', found, mortTrouvee)
244
245     m2 =
246     m1 { mEnnemis = enns2
247     , mProjectiles = projsRestes
248     , mScores = scoresGagnes
249     , mScore = scoreTotal scoresGagnes
250     }
251     (meteoiresRestants, jousApresMeteores) =
252     foldr appliquerMeteoreSurJoueuses ([], mJoueuses m2) (mMeteores m2)
253
254     appliquerMeteoreSurJoueuses mt (acc, js) =
255     let (js', touche) = parcourirJoueusesMeteore mt js
256     in if touche
257     then (acc, js')
258     else (mt : acc, js')
259
260     parcourirJoueusesMeteore _ [] =
261     ([], False)
262
263     parcourirJoueusesMeteore mt (j:js) =
264     let (j', hit) = joueuseToucheeMeteore j mt
265     in if hit
266     then (supprimerJoueusesEliminees (j' : js), True)
267     else
268     let (js'', found) = parcourirJoueusesMeteore mt js
269     in (j : js'', found)
270
271     m3 =
272     m2 { mMeteores = meteoiresRestants
273     , mJoueuses = jousApresMeteores
274     }
275
276     jous3 =
277     [ foldl joueuseToucheeObstacle j (mObstacles m3)
278     | j <- mJoueuses m3
279     ]
280
281     jous4 =
282     supprimerJoueusesEliminees $
283     map (joueuseToucheeMurs (mMurs m3)) jous3
284
285     in m3 { mJoueuses = jous4 }

```

Listing 3 – Module Collisions.hs : résolution des impacts et stabilisation des slots joueurs